



UNIVERSIDADE
LUSÓFONA



ReStock - Aplicação Inteligente de Gestão de Compras e Inventário Doméstico

**Licenciatura em Engenharia Informática
Projeto I & II**

Relatório

1º e 2º

Semestre

Hugo Emanuel Pereira Moreira, a22402246

Orientador: Hugo Barbosa

Departamento de Engenharia Informática e Sistemas de Informação
Universidade Lusófona, Centro Universitário do Porto (CUP)

Setembro 2025

www.ulusofona.pt

Direitos de cópia

ReStock, Copyright de Hugo Emanuel Pereira Moreira, Universidade Lusófona.

A Faculdade de Ciências Naturais, Engenharia e Tecnologias (FCNET) e a Universidade Lusófona têm o direito, perpétuo e sem limites geográficos, de arquivar e publicar esta dissertação/monografia através de exemplares impressos reproduzidos em papel ou de forma digital, ou por qualquer outro meio conhecido ou que venha a ser inventado, e de a divulgar através de repositórios científicos e de admitir a sua cópia e distribuição com objetivos educacionais ou de investigação, não comerciais, desde que seja dado crédito ao autor e editor.

Agradecimentos

Agradeço ao Professor Hugo Barbosa pela orientação, disponibilidade e apoio prestados ao longo de todo o desenvolvimento deste projeto. O seu acompanhamento foi essencial para manter o foco e a qualidade do trabalho realizado. Agradeço também à minha família, pelo suporte incondicional, e aos colegas que participaram nos testes da aplicação, contribuindo com o seu tempo e feedback para a melhoria da solução.

Resumo

O projeto **ReStock - Aplicação Inteligente de Gestão de Compras e Inventário Doméstico** tem como principal objetivo apoiar famílias e indivíduos na gestão dos produtos alimentares existentes em casa, de forma simples e eficiente. A aplicação permite o registo de produtos de várias formas, nomeadamente de modo manual, através da leitura de códigos de barras ou pela digitalização de faturas recorrendo a OCR, com recurso ao **Google ML Kit**.

Os dados são armazenados e sincronizados em tempo real através do **Firebase Firestore**, o que possibilita a partilha de inventários e listas de compras entre os diferentes membros da mesma família. Para além disso, a aplicação disponibiliza notificações automáticas para produtos próximos do prazo de validade, controlo do orçamento mensal e um sistema de gamificação que incentiva a poupança e a redução do desperdício alimentar.

O desenvolvimento da aplicação foi realizado em **Kotlin**, no ambiente **Android Studio**, utilizando o **Firebase** como base de dados principal. Este relatório apresenta as diferentes etapas de planeamento e conceção da aplicação, descrevendo a sua arquitetura, os requisitos definidos, a modelação efetuada e o método de desenvolvimento adotado.

Palavras-chave: gestão de inventário; sustentabilidade; aplicação móvel; Firebase; OCR

Abstract

The **ReStock - Intelligent Shopping and Home Inventory Management Application** aims to help families and individuals manage household products efficiently. The app allows users to register products manually, by barcode scanning or invoice recognition through OCR, using **Google ML Kit**. Data is stored and synchronized in real time via **Firebase Firestore**, ensuring shared inventory and shopping lists among family members. The application also provides automatic notifications for expiring products, budget control, and a gamification system to encourage savings and reduce food waste. The project was developed in **Kotlin** using **Android Studio**, with **Firebase** as the main database. This report presents the planning and design stages, including the architecture, requirements, modelling, and methodology.

Keywords: inventory management; sustainability; mobile app; Firebase; OCR.

Índice

Agradecimentos.....	3
Resumo.....	4
Abstract.....	5
Índice.....	6
Lista de Figuras	8
Lista de Tabelas	9
Lista de Siglas	10
1 Introdução	11
1.1 Enquadramento.....	11
1.2 Motivação e Identificação do Problema.....	11
1.3 Objetivos.....	12
1.4 Estrutura do Documento.....	12
2 Pertinência e Viabilidade.....	1
2.1 Pertinência.....	1
Validação através de Inquérito	1
Síntese dos Resultados.....	4
2.2 Viabilidade	5
2.3 Análise Comparativa com soluções existentes	6
2.3.1 Soluções existentes.....	6
2.3.2 Análise de benchmarking	6
2.4 Proposta de inovação e mais-valias.....	6
2.5 Identificação de oportunidade de negócio.....	6
3 Especificação e Modelação.....	7
3.1 Análise de requisitos.....	7
3.1.1 Requisitos funcionais	7
3.1.2 Requisitos não-funcionais.....	8
3.1.3 Casos de Uso	10
3.2 Modelação.....	12
3.2.1 Diagrama Entidade - Relação (E-R).....	12
3.2.2 Modelo de Classes (UML).....	13
3.2.3 Diagramas de Atividade (Activity Diagram).....	13
3.3 Protótipos de Interface	16

4	Solução Desenvolvida	17
4.1	Introdução.....	17
4.2	Arquitetura	17
4.3	Tecnologias e Ferramentas Utilizadas	19
4.4	Ambientes de Teste e de Produção.....	21
4.5	Abrangência.....	21
4.6	Componentes.....	22
4.6.1	Componente 1: Aplicação móvel Android	22
4.6.2	Componente 2: Backend e persistência de dados em cloud	22
4.6.3	Componente 3: Leitura de código de barras e obtenção automática de dados	22
4.6.4	Componente 4: Notificações e gestão local de preferências	22
4.7	Interfaces.....	23
5	Testes e Validação	29
5.1	Abordagem e Justificação	29
5.2	Análise de Risco e Impacto.....	29
5.3	Validação de Qualidade Técnica e Funcional	29
5.4	Validação Operacional e Cloud.....	30
5.5	Validação em Contexto Real e Participação de Terceiros.....	30
5.6	Conclusão da Validação	30
6	Método e Planeamento	31
7	Resultados	33
7.1	Resultados dos testes	33
7.2	Cumprimento de requisitos.....	33
8	Conclusão	34
	Trabalhos Futuros.....	34
9	Referências bibliográficas	35

Lista de Figuras

Figura 1 - Distribuição etária dos inquiridos.	1
Figura 2 - Situação habitacional dos inquiridos.	2
Figura 3 - Frequência de esquecimento de produtos alimentares fora do prazo de validade.....	2
Figura 4 - Compra de produtos repetidos por falta de controlo do inventário doméstico.	3
Figura 5 - Perceção da utilidade de uma aplicação móvel para gestão de produtos alimentares e listas de compras.....	3
Figura 6 - Funcionalidades consideradas mais relevantes numa aplicação de gestão de inventário doméstico.....	4
Figura 7 - Intenção de utilização da aplicação ReStock no dia a dia.	4
Figura 8 - Diagrama de Casos de Uso da aplicação ReStock.....	11
Figura 9 - (Utilizador) Partilha Familiar.	11
Figura 10 - (Utilizador) Receber Notificações de Validade.....	12
Figura 11 - Diagrama Entidade-Relação.....	12
Figura 12 - Modelo de Classes.	13
Figura 13 - Diagrama de Atividade Autenticação de Utilizador.	14
Figura 14 - Diagrama de Atividade Adicionar Produto.	14
Figura 15 - Diagrama de Atividade Receber Notificações de Validade.....	15
Figura 16 - Diagrama de Atividade Gerir Listas de Compras.	15
Figura 17 - Diagrama de Atividade Sincronização Familiar.	16
Figura 18 - Diagrama de Componentes Arquitetura MVVM.....	18
Figura 19 - Diagrama de Arquitetura em Camadas.....	20
Figura 20 Milestones.....	31
Figura 21 Diagrama de Gantt.....	32

Lista de Tabelas

Tabela 1 - SWOT Analysis	5
Tabela 2 - Tabela de Benchmarking.....	6
Tabela 3 - Requisitos Funcionais	7
Tabela 4 - Requisitos Não Funcionais	8
Tabela 5 - Atores do Sistema	10
Tabela 6 - Casos de Uso (Utilizador)	10
Tabela 7 - Casos de Uso (Partilha Familiar)	10
Tabela 8 - Sistema Externo (Firebase).....	10
Tabela 9 - Ferramentas de Desenvolvimento.....	20
Tabela 10 - Análise de Risco e Impacto.....	29
Tabela 11 - Estado de cumprimento de Requisitos funcionais.....	33
Tabela 12 - A1. Testes Unitários Automatizados	36
Tabela 13 - A2. Testes de Interface (Espresso)	36
Tabela 14 - A3. Testes de Validação em Contexto Real.....	37

Lista de Siglas

API	Interface de Programação de Aplicações
HTML	Linguagem de Marcação de Hipertexto
OCR	Optical Character Recognition
ODS	Objetivos de Desenvolvimento Sustentável
UML	Unified Modeling Language
ER	Entidade–Relação
RGPD	Regulamento Geral sobre a Proteção de Dados

1 Introdução

1.1 Enquadramento

Atualmente, a gestão de compras e de produtos alimentares é uma das tarefas mais comuns no quotidiano das famílias, mas também uma das mais desorganizadas. Muitos consumidores acabam por adquirir produtos em duplicado, desperdiçar alimentos por falta de controlo dos prazos de validade ou ultrapassar o orçamento mensal devido a uma gestão pouco eficaz do inventário doméstico.

A transformação digital e o acesso generalizado a dispositivos móveis vieram criar oportunidades para melhorar este tipo de gestão, através de aplicações inteligentes que automatizam tarefas e apoiam a tomada de decisões.

Neste contexto, surgiu a ideia de desenvolver a ReStock, uma aplicação móvel criada em **Kotlin**, que utiliza o **Firestore** como principal infraestrutura de dados e o **ML Kit** para a leitura de códigos de barras e para o reconhecimento **óptico de caracteres (OCR)**. A aplicação tem como objetivo simplificar a forma como as famílias gerem os seus produtos alimentares, listas de compras e despesas, promovendo simultaneamente hábitos de consumo mais sustentáveis e conscientes. A ReStock distingue-se por integrar, num único sistema, funcionalidades que habitualmente se encontram distribuídas por várias aplicações, tais como o registo automático de produtos, notificações de validade, controlo financeiro e partilha de inventários entre utilizadores.

1.2 Motivação e Identificação do Problema

O desperdício alimentar é uma preocupação cada vez mais relevante, tanto do ponto de vista ambiental como económico. Em muitos lares, uma parte significativa dos alimentos acaba por ser desperdiçada devido à falta de controlo sobre os produtos existentes e ao esquecimento das respetivas datas de validade.

A principal motivação para o desenvolvimento deste projeto foi a criação de uma aplicação que contribua para a redução deste problema, oferecendo aos utilizadores uma forma prática e simples de saber o que têm em casa, quando os produtos expiram e o que realmente precisam de comprar. Para além da vertente ambiental, existe também uma motivação de carácter pessoal e social, relacionada com a simplificação da gestão do quotidiano. A aplicação permite que famílias, casais ou colegas de casa partilhem listas de compras e inventários em tempo real. O problema identificado centra-se, assim, na inexistência de uma ferramenta completa, acessível e gratuita que reúna estas funcionalidades de forma integrada, intuitiva e fácil de utilizar.

1.3 Objetivos

O objetivo principal do projeto é desenvolver uma aplicação móvel inteligente que auxilie na **gestão de inventário e compras domésticas**, minimizando o desperdício alimentar e promovendo a sustentabilidade.

Objetivos específicos:

- Permitir o registo de produtos através de código de barras e leitura de faturas (OCR).
- Armazenar e sincronizar dados em tempo real utilizando o Firebase Firestore.
- Alertar o utilizador sobre produtos prestes a expirar.
- Gerar listas de compras automáticas e inteligentes.
- Implementar partilha familiar de inventários e listas.
- Incluir um sistema de gamificação que incentive boas práticas de consumo.

1.4 Estrutura do Documento

Este relatório está organizado em oito capítulos:

- No **Capítulo 1** é apresentada a introdução ao tema, a motivação e os objetivos do projeto.
- O **Capítulo 2** aborda a pertinência e a viabilidade da solução, incluindo a validação junto do público-alvo e a análise comparativa com soluções existentes.
- O **Capítulo 3** detalha a especificação e a modelação da aplicação, apresentando os requisitos funcionais e não-funcionais, os casos de uso e os diagramas de atividade.
- O **Capítulo 4** descreve a solução desenvolvida, incluindo a arquitetura, as tecnologias utilizadas, os componentes implementados e as interfaces da aplicação.
- O **Capítulo 5** apresenta a estratégia de testes e validação da solução, contemplando testes unitários, testes de interface e validação em contexto real.
- O **Capítulo 6** descreve o método e o planeamento do projeto ao longo dos dois semestres.
- O **Capítulo 7** apresenta os resultados obtidos e o grau de cumprimento dos requisitos definidos.
- O **Capítulo 8** apresenta as conclusões do projeto e os trabalhos futuros previstos.

O Capítulo 5 apresenta os testes e a validação da solução. O Capítulo 6 descreve o método e o planeamento do projeto. O Capítulo 7 apresenta os resultados obtidos. O Capítulo 8 apresenta as conclusões e os trabalhos futuros.

ReStock - Aplicação Inteligente de Gestão de Compras e Inventário Doméstico

O quadro abaixo é indicativo de entregáveis (*deliverables*/relatórios) para cada momento de avaliação, assumindo abordagem sequencial do desenvolvimento do trabalho de final de curso (TFC), incorporando as unidades curriculares de **Projeto I** (1º semestre) e **Projeto II** (2º semestre). Podendo ser adotadas outras abordagens metodológicas (e.g.: metodologias ágeis), serão aceites outras organizações de entregas no âmbito do desenvolvimento das UCs Projeto I e Projeto II. Deve-se, no entanto, observar duas condições: (i) as entregas deverão manter os conteúdos indicados no quadro, por forma a permitir ao júri (professores) avaliar a pertinência do tema e a taxa de esforço esperada; (ii) a organização de conteúdos em cada entrega deve ter atenção aos critérios de avaliação de modo a garantir a uniformidade da avaliação das UCs.

Quadro de conteúdos – desenvolvimento (Projeto I e Projeto II)

Avaliação	Tipo	1. Introdução	2. Pertinência e Viabilidade	3. Especificação e Modelação	4. Solução (arquitetura) desenvolvida ¹	5. Testes e validação	6. Método e planeamento	7. Resultados	8. Conclusão
Entrega Intercalar Projeto I (Nov 2025)	Qualitativa	Incluir	Incluir	Incluir	Opcional	N/A	Incluir	Opcional	Opcional
Entrega relatório final Projeto I (Jan 2026)	Quantitativa	Incluir	Incluir	Incluir	Opcional	N/A	Incluir	Opcional	Incluir (no âmbito de Projeto I)
Entrega Intercalar Projeto II (Abril 2026)	Qualitativa	Incluir	Incluir	Incluir	Incluir	Incluir	Incluir	Opcional	Opcional
Entrega relatório final² Projeto II (Maio 2026)	Quantitativa	Final	Final	Final	Final	Final	Final ³	Final	Final

¹ Sempre que aplicável, inclui código funcional, a disponibilizar em repositório *Git*

² O relatório final deverá incluir todos os conteúdos desenvolvidos ao longo do TFC (Projeto I e Projeto II)

³ Para a entrega final sugere-se a inclusão, em anexo, de todo os planos de trabalho anteriores com apreciação de progresso e avaliação das dificuldades encontradas na elaboração e cumprimento do planeamento

2 Pertinência e Viabilidade

2.1 Pertinência

O ReStock apresenta uma solução com impacto direto na redução do desperdício alimentar e na otimização da gestão doméstica. Ao permitir o controlo dos produtos em tempo real e ao disponibilizar alertas automáticos, a aplicação promove um consumo mais racional, consciente e sustentável.

Para além disso, integra funcionalidades que não são comuns em muitas aplicações semelhantes, como o registo automático de produtos através da digitalização de faturas por OCR e a partilha do inventário entre membros da mesma família.

Do ponto de vista social, a aplicação pode contribuir para a poupança de tempo e dinheiro das famílias, ao reduzir compras desnecessárias e incentivar um melhor planeamento alimentar. A relevância do projeto é ainda reforçada pelo seu alinhamento com os Objetivos de Desenvolvimento Sustentável (ODS) das Nações Unidas, em particular o ODS 12 (Consumo e Produção Sustentáveis).

Validação através de Inquérito

Com o objetivo de validar a pertinência da aplicação **ReStock** junto do público-alvo, foi realizado um inquérito online através da plataforma Google Forms. O inquérito permitiu recolher dados sobre hábitos de consumo, gestão de produtos alimentares e interesse na utilização de uma aplicação móvel de apoio à gestão de inventário doméstico.

O inquérito obteve **54 respostas válidas**, o que permite retirar conclusões relevantes para o contexto do projeto.

Relativamente à faixa etária dos inquiridos, verifica-se uma predominância de participantes entre os **18 e os 25 anos (50%)**, seguindo-se o grupo dos **36 aos 50 anos (22,2%)**. Os restantes participantes distribuem-se pelas faixas etárias dos **26 aos 35 anos, menos de 18 anos (9,3%)** e **mais de 50 anos (11,1%)**. Estes dados indicam uma amostra diversificada, com forte presença de utilizadores ativos de tecnologias móveis.

Qual a sua faixa etária?

 Copy chart

54 responses

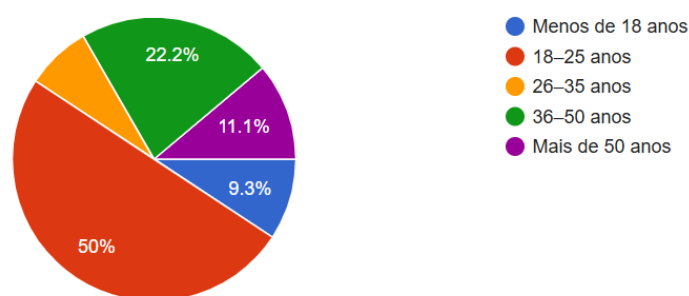


Figura 1 - Distribuição etária dos inquiridos.

No que diz respeito à situação habitacional, a maioria dos inquiridos **vive em família (77,8%)**, seguida de participantes que vivem **em casal (14,8%)**, sendo residual a percentagem de pessoas que vivem sozinhas ou com colegas de casa. Este resultado é particularmente relevante, uma vez que a aplicação ReStock foi pensada para contextos de utilização partilhada e gestão familiar.

Vive atualmente:

 [Copy chart](#)

54 responses

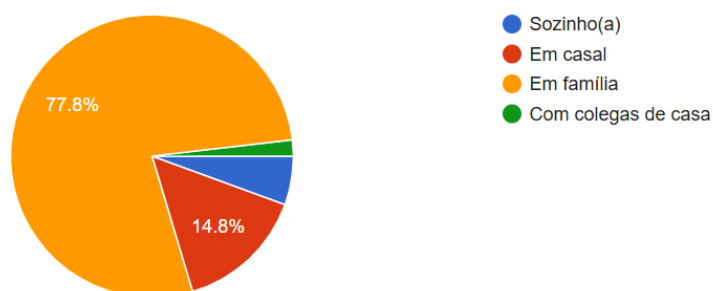


Figura 2 - Situação habitacional dos inquiridos. .

Os resultados do inquérito revelam que a gestão de produtos alimentares é uma dificuldade comum. Quando questionados sobre o esquecimento de produtos que acabam por ultrapassar a data de validade, **37,7% dos inquiridos indicaram que isso acontece “às vezes”** e **13,2% afirmaram que acontece “muitas vezes”**. Apenas **15,1% referiram nunca passar por essa situação**, demonstrando que o desperdício alimentar é um problema recorrente.

Costuma esquecer-se de produtos alimentares que acabam por passar da validade?

 [Copy chart](#)

53 responses

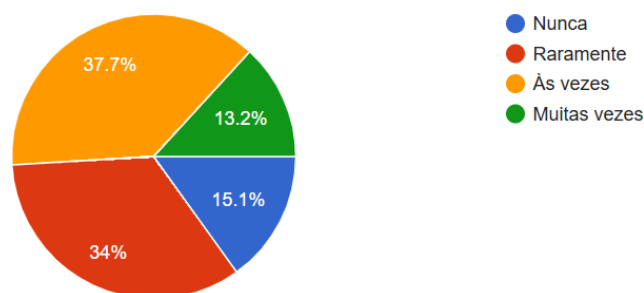


Figura 3 - Frequência de esquecimento de produtos alimentares fora do prazo de validade.

Adicionalmente, **85,2% dos participantes afirmaram já ter comprado produtos repetidos por não saberem que já os tinham em casa**, o que evidencia a falta de controlo do inventário doméstico e reforça a necessidade de uma solução digital de apoio à gestão de compras.

Já comprou produtos repetidos por não saber que já os tinha em casa?

 [Copy chart](#)

54 responses

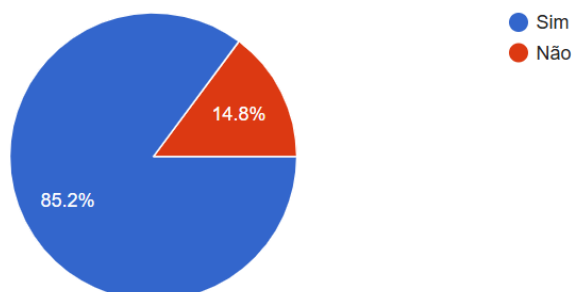


Figura 4 - Compra de produtos repetidos por falta de controlo do inventário doméstico.

Quando questionados sobre a utilidade de uma aplicação móvel que ajude a gerir produtos alimentares e listas de compras, **51,9% dos inquiridos consideraram a aplicação “útil” e 38,9% classificaram-na como “muito útil”**. Apenas uma pequena percentagem considerou a aplicação pouco útil, não se registando respostas que a classificassem como nada útil.

Consideraria útil uma aplicação móvel que ajude a gerir produtos alimentares e listas de compras?

 [Copy chart](#)

54 responses

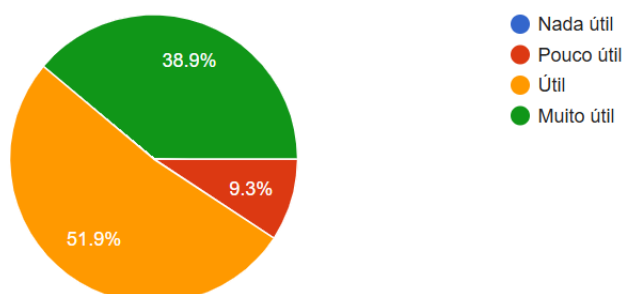


Figura 5 - Perceção da utilidade de uma aplicação móvel para gestão de produtos alimentares e listas de compras.

Relativamente às funcionalidades mais relevantes numa aplicação deste tipo, destacam-se a **criação automática de listas de compras (42,6%)** e os **alertas de validade (31,5%)**, seguidos da **leitura de códigos de barras (13%)** e da **digitalização de faturas através de OCR (7,4%)**. A **partilha familiar**, apesar de apresentar uma percentagem inferior, continua a ser valorizada num contexto de utilização doméstica partilhada.

Quais funcionalidades considera mais relevantes numa aplicação deste tipo?

 [Copy chart](#)

54 responses

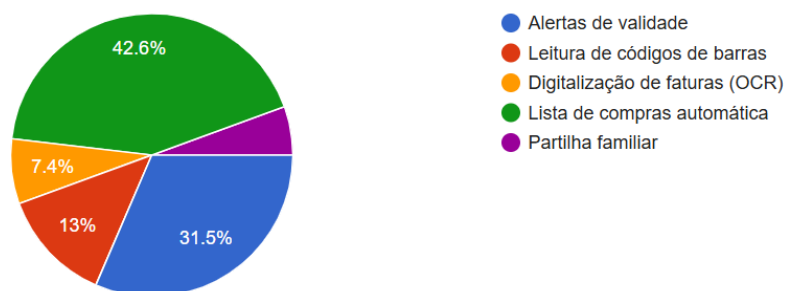


Figura 6 - Funcionalidades consideradas mais relevantes numa aplicação de gestão de inventário doméstico.

Por fim, quando questionados diretamente sobre a utilização da aplicação ReStock no dia a dia, **63% dos inquiridos responderam afirmativamente**, enquanto **27,8% indicaram que talvez utilizariam**. Apenas **9,3% afirmaram que não utilizariam** a aplicação, o que demonstra uma elevada predisposição para a adoção da solução proposta.

Utilizaria uma aplicação como o ReStock no seu dia a dia?

 [Copy chart](#)

54 responses

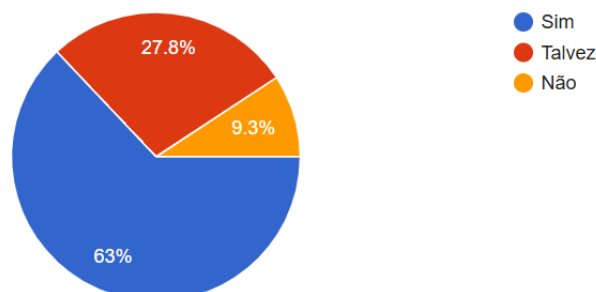


Figura 7 - Intenção de utilização da aplicação ReStock no dia a dia.

Síntese dos Resultados

Os resultados do inquérito confirmam a **pertinência da aplicação ReStock**, demonstrando que existe uma necessidade real no público-alvo por ferramentas que auxiliem a gestão de inventário doméstico, reduzam o desperdício alimentar e evitem compras desnecessárias. A elevada taxa de aceitação da aplicação e a valorização das funcionalidades propostas validam a relevância do projeto e justificam o seu desenvolvimento no âmbito do Projeto I.

2.2 Viabilidade

A viabilidade técnica do projeto é elevada, uma vez que recorre a ferramentas gratuitas, modernas e acessíveis para estudantes e pequenos projetos.

O Firebase é a base de dados escolhida por permitir sincronização em tempo real, autenticação de utilizadores, armazenamento de imagens e envio de notificações push, numa plataforma acessível e com suporte a escalabilidade progressiva.

O desenvolvimento em **Kotlin** no **Android Studio** garante compatibilidade com a maioria dos dispositivos Android, permitindo uma implementação e testes simples.

A nível económico, o projeto é sustentável, pois não requer investimento em servidores nem licenças. Os custos são essencialmente de tempo de desenvolvimento e manutenção. No futuro, a aplicação poderá evoluir para um modelo de negócio com integração de APIs de supermercados, programas de fidelização ou publicidade sustentável.

Tabela 1 - SWOT Analysis

<i>Strengths</i>	<i>Weaknesses</i>	<i>Opportunities</i>	<i>Threats</i>
Projeto com um tema atual e relevante (redução do desperdício alimentar e sustentabilidade).	Desenvolvimento individual, o que pode limitar o ritmo de progresso.	Aplicações de sustentabilidade e gestão doméstica estão em crescimento.	Existência de aplicações semelhantes no mercado internacional.
Integração com IA (OCR e gamificação) e sincronização em tempo real.	Experiência limitada no desenvolvimento de aplicações móveis completas.	Possibilidade de evolução futura (versão multiplataforma)	Possíveis dificuldades técnicas na integração com APIs externas.
Experiência fluida offline/online.		Potencial para evolução futura em contexto profissional.	Possíveis dificuldades técnicas na integração da inteligência artificial na aplicação.

2.3 Análise Comparativa com soluções existentes

2.3.1 Soluções existentes

Existem algumas aplicações semelhantes no mercado, como **NoWaste**, **Pantry Check** e **Out of Milk**, que permitem gerir inventários e listas de compras.

No entanto, estas aplicações apresentam limitações: a maioria é paga, não possui leitura de faturas, nem funcionalidades de partilha familiar. Algumas delas não são traduzidas para português e não têm ligação com os hábitos e contextos dos consumidores portugueses.

2.3.2 Análise de benchmarking

Tabela 2 - Tabela de Benchmarking.

Funcionalidade	NoWaste	Pantry Check	Out of Milk	ReStock
Registo manual de produtos	X	X	X	X
Leitura de códigos de barras	X	X		X
Leitura de faturas (OCR)				X
Alertas automáticos de validade	X	X	X	X
Partilha entre utilizadores		X (limitado)		X
Gamificação				X
Gratuito	Parcial		X	X

A análise mostra que o **ReStock** se destaca por integrar num só sistema funcionalidades de leitura automática, partilha familiar e gamificação, tudo gratuito.

2.4 Proposta de inovação e mais-valias

O ReStock é inovador porque combina **inteligência artificial leve (OCR)**, **sincronização em tempo real (Firebase)** e **gamificação** numa única aplicação móvel.

As principais mais-valias são:

- Integração de múltiplas funcionalidades num sistema gratuito e acessível.
- Incentivo à sustentabilidade e à poupança doméstica.
- Experiência fluida e intuitiva, adequada a qualquer utilizador.
- Escalabilidade técnica para crescimento futuro.

2.5 Identificação de oportunidade de negócio

A aplicação tem potencial de expansão comercial.

No futuro, poderá incluir integração com supermercados nacionais, sincronização de recibos eletrónicos e sugestões personalizadas de promoções.

Além disso, o modelo de partilha familiar pode ser aproveitado para desenvolver versões empresariais, voltadas para pequenas mercearias ou grupos de consumo colaborativo.

3 Especificação e Modelação

3.1 Análise de requisitos

A análise de requisitos permite definir as funcionalidades e as características que a aplicação deve apresentar, garantindo que responde adequadamente ao problema identificado e aos objetivos definidos.

A aplicação ReStock foi concebida para ser intuitiva, acessível e eficiente, com suporte para utilização partilhada e sincronização em tempo real.

3.1.1 Requisitos funcionais

Os requisitos funcionais descrevem o que a aplicação deve fazer.

Para o ReStock, foram identificados os seguintes:

Tabela 3 - Requisitos Funcionais

ID	Requisito Funcional	Prioridade	Motivação	Descrição
RF01	Autenticação de Utilizadores	Alta	Garantir acesso seguro e personalizado à aplicação	O sistema deve permitir o registo e login de utilizadores através de email e palavra-passe, utilizando o Firebase Authentication.
RF02	Gestão de Perfil	Média	Permitir personalização da experiência do utilizador	O utilizador deve poder visualizar e atualizar os seus dados pessoais e fotografia de perfil.
RF03	Adicionar Produto Manualmente	Alta	Permitir controlo direto do inventário doméstico	O utilizador deve conseguir adicionar produtos ao inventário introduzindo manualmente os dados necessários.
RF04	Leitura de Código de Barras	Alta	Facilitar e acelerar o registo de produtos	A aplicação deve permitir a leitura de códigos de barras para preenchimento automático dos dados do produto.
RF05	Digitalização de Faturas (OCR)	Alta	Automatizar a adição de produtos ao inventário	O sistema deve identificar automaticamente produtos presentes em faturas através de OCR e adicioná-los ao inventário.
RF06	Atualizar e Remover Produtos	Alta	Manter o inventário atualizado e correto	O utilizador deve poder editar ou eliminar produtos existentes no inventário.
RF07	Notificações de Validade	Alta	Reduzir o desperdício alimentar	O sistema deve gerar alertas automáticos quando um produto se encontra próximo da data de validade.
RF08	Criação Automática de Lista de Compras	Média	Apoiar o planeamento eficiente das compras	A aplicação deve gerar listas de compras automaticamente com base em produtos em falta ou prestes a expirar.
RF09	Gestão Manual da Lista de Compras	Média	Oferecer flexibilidade na organização das compras	O utilizador deve poder criar, editar e marcar itens da lista de compras como comprados.
RF10	Partilha Familiar	Alta	Permitir colaboração entre membros do agregado familiar	O sistema deve permitir a criação de grupos familiares e a partilha de inventários e listas de compras em tempo real.

RF11	Sincronização em Tempo Real	Alta	Garantir informação atualizada entre todos os utilizadores	Os dados devem ser sincronizados em tempo real entre os dispositivos associados à mesma família através do Firebase Firestore.
RF12	Recomendações de Receitas	Baixa	Incentivar o aproveitamento dos produtos disponíveis	A aplicação deve sugerir receitas com base nos produtos existentes no inventário.
RF13	Gamificação	Baixa	Motivar o utilizador para boas práticas de consumo	O sistema deve atribuir pontos ou conquistas associadas à redução de desperdício alimentar.
RF14	Comparação de Preços	Baixa	Apoiar decisões de compra mais económicas	A aplicação deve permitir a comparação de preços de produtos entre supermercados através de APIs externas.
RF15	Armazenamento de Imagens	Média	Melhorar a identificação visual dos produtos	As imagens dos produtos devem ser armazenadas no Firebase Storage e associadas ao inventário.

3.1.2 Requisitos não-funcionais

Os requisitos não-funcionais definem as propriedades de qualidade do sistema.

Para a aplicação ReStock, destacam-se:

Tabela 4 - Requisitos Não Funcionais

ID	Requisito Não Funcional	Prioridade	Motivação	Descrição
RNF01	Interface Intuitiva	Alta	Facilitar a utilização por utilizadores com diferentes níveis de literacia digital	A interface deve ser simples, clara e seguir padrões de design consistentes, adequados a aplicações móveis.
RNF02	Consistência Visual	Média	Garantir uma experiência de utilização coerente em toda a aplicação	A aplicação deve manter cores, tipografia e ícones uniformes em todos os ecrãs.
RNF03	Usabilidade Móvel	Alta	Adaptar à aplicação ao uso frequente em smartphones	A aplicação deve ser otimizada para utilização em dispositivos móveis Android, com navegação fluida e responsiva.
RNF04	Acessibilidade	Média	Tornar a aplicação acessível a um maior número de utilizadores	A aplicação deve suportar tamanhos de texto ajustáveis, contraste adequado e compatibilidade com leitores de ecrã.
RNF05	Proteção de Dados (RGPD)	Alta	Proteger a privacidade dos utilizadores	Os dados pessoais devem ser tratados de acordo com o RGPD, garantindo confidencialidade e utilização responsável.
RNF06	Armazenamento Seguro	Alta	Evitar acessos não autorizados aos dados	Os dados devem ser armazenados de forma segura no Firebase, com regras de acesso bem definidas.
RNF07	Autenticação Segura	Alta	Garantir que apenas utilizadores autorizados acedem à aplicação	O acesso deve ser protegido através de autenticação segura com Firebase Authentication.

RNF08	Sincronização Fiável	Alta	Garantir consistência dos dados partilhados	A sincronização dos dados entre membros do mesmo grupo familiar deve ocorrer de forma fiável e consistente.
RNF09	Disponibilidade	Alta	Assegurar acesso contínuo à aplicação	A aplicação deve garantir elevada disponibilidade, suportada pela infraestrutura cloud do Firebase.
RNF10	Persistência de Dados	Alta	Evitar perda de informação do utilizador	Os dados do inventário e listas de compras devem ser armazenados de forma persistente, mesmo após encerramento da aplicação.
RNF11	Portabilidade	Média	Permitir utilização em vários dispositivos Android	A aplicação deve ser compatível com Android 10 (API Level 29) ou superior.
RNF12	Manutenibilidade	Média	Facilitar manutenção e evolução futura	O código deve ser modular e organizado segundo boas práticas de desenvolvimento.

3.1.3 Casos de Uso

Os casos de uso representam as principais interações entre os utilizadores e a aplicação ReStock, descrevendo de forma estruturada as funcionalidades disponibilizadas pelo sistema. Estes casos de uso foram definidos com base nos requisitos funcionais identificados anteriormente e servem de apoio à modelação e implementação da aplicação.

Tabela 5 - Atores do Sistema

ATOR	DEFINIÇÃO
UTILIZADOR	Entidade principal que utiliza a aplicação para gerir o inventário doméstico, listas de compras e funcionalidades associadas.
MEMBRO DA FAMÍLIA	Utilizador pertencente a um grupo familiar, com acesso partilhado ao inventário e listas de compras.
SISTEMA FIREBASE	Serviço externo responsável pela autenticação, armazenamento e sincronização dos dados.

Tabela 6 - Casos de Uso (Utilizador)

CASO DE USO	DEFINIÇÃO
AUTENTICAR UTILIZADOR	Permite ao utilizador registar-se e efetuar login na aplicação através de credenciais válidas.
GERIR PERFIL	Permite ao utilizador consultar e atualizar os seus dados pessoais e fotografia de perfil.
ADICIONAR PRODUTO	Permite adicionar produtos ao inventário de forma manual, por código de barras ou através de OCR.
ATUALIZAR OU REMOVER PRODUTO	Permite editar ou eliminar produtos existentes no inventário.
CONSULTAR INVENTÁRIO	Permite visualizar a lista de produtos disponíveis e respetivas informações.
RECEBER NOTIFICAÇÕES DE VALIDADE	Permite ao utilizador receber alertas sobre produtos próximos da data de validade.
GERIR LISTA DE COMPRAS	Permite criar, editar e marcar itens da lista de compras como comprados.
CONSULTAR SUGESTÕES DE RECEITAS	Permite visualizar receitas sugeridas com base nos produtos disponíveis.

Tabela 7 - Casos de Uso (Partilha Familiar)

CASO DE USO	DEFINIÇÃO
CRIAR GRUPO FAMILIAR	Permite criar um grupo familiar para partilha de inventário.
PARTILHAR INVENTÁRIO	Permite que os membros do grupo tenham acesso ao mesmo inventário.
SINCRONIZAR DADOS EM TEMPO REAL	Garante que as alterações efetuadas por um membro são refletidas nos restantes dispositivos.

Tabela 8 - Sistema Externo (Firebase)

CASO DE USO	DEFINIÇÃO
AUTENTICAÇÃO DE UTILIZADORES	Valida as credenciais dos utilizadores através do Firebase Authentication.
SINCRONIZAÇÃO DE DADOS	Armazena e sincroniza os dados do inventário e listas de compras em tempo real.

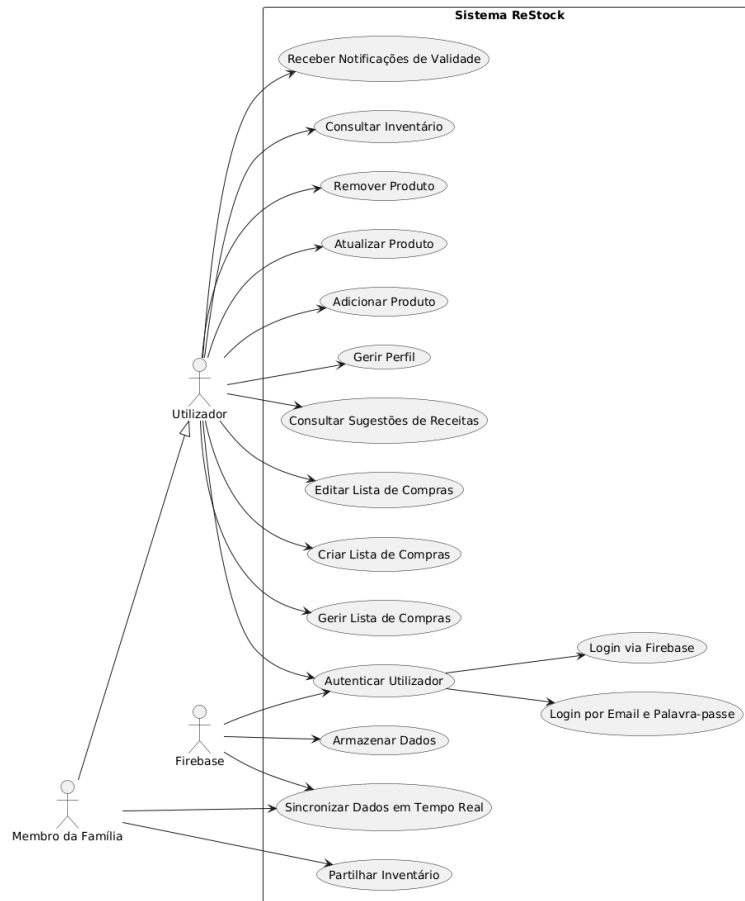


Figura 8 - Diagrama de Casos de Uso da aplicação ReStock.

O diagrama de casos de uso apresenta as principais interações entre os utilizadores e a aplicação ReStock, evidenciando as funcionalidades disponíveis para a gestão de inventário doméstico, listas de compras e partilha familiar. O diagrama inclui ainda a interação com o sistema externo Firebase, responsável pela autenticação, armazenamento e sincronização dos dados.

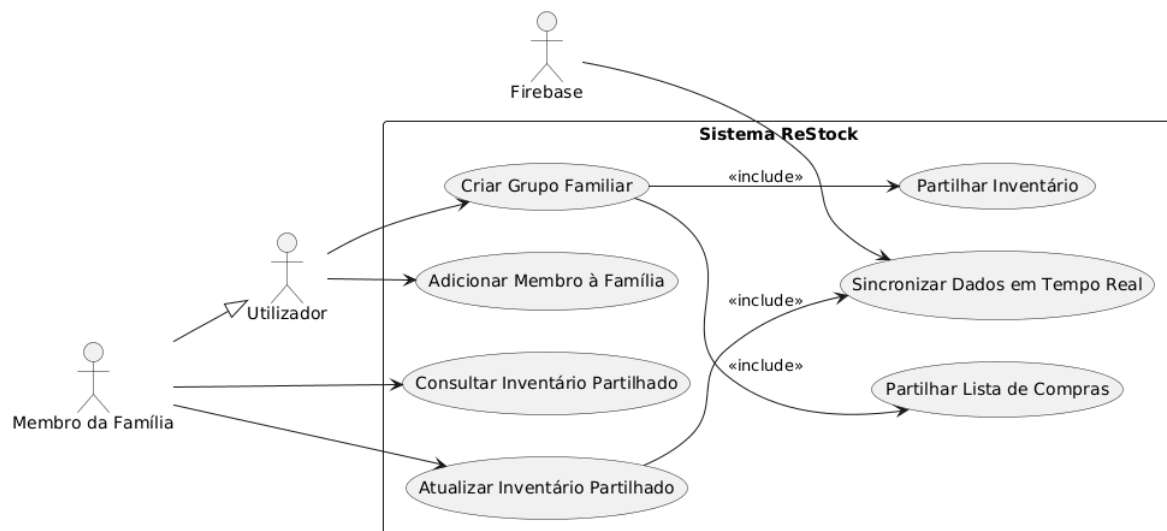


Figura 9 - (Utilizador) Partilha Familiar.

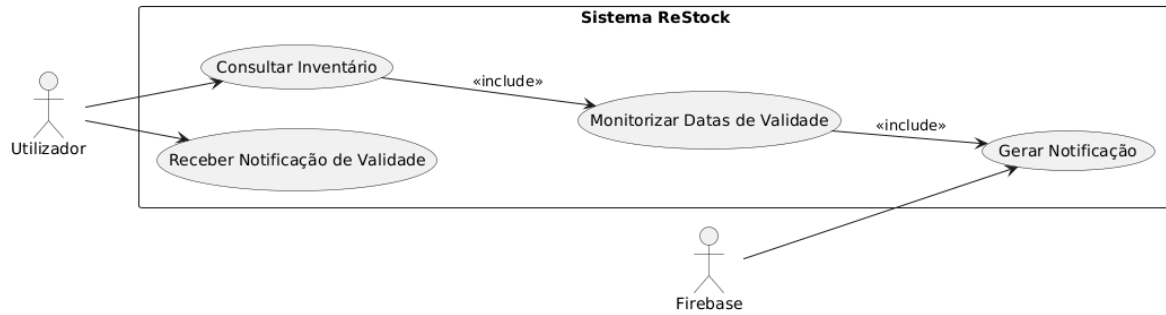


Figura 10 - (Utilizador) Receber Notificações de Validade.

3.2 Modelação

A modelação de dados da aplicação ReStock foi projetada de forma a garantir eficiência, escalabilidade e sincronização em tempo real entre utilizadores.

Apesar do Firebase Firestore ser uma base de dados NoSQL, o modelo lógico foi estruturado com base nos princípios relacionais tradicionais, o que facilita a compreensão e documentação do sistema.

Cada coleção do Firestore equivale a uma tabela relacional, e cada documento representa um registo (linha) dessa tabela.

Os relacionamentos são estabelecidos através de chaves de referência (IDs), simulando foreign keys.

3.2.1 Diagrama Entidade - Relação (E-R)

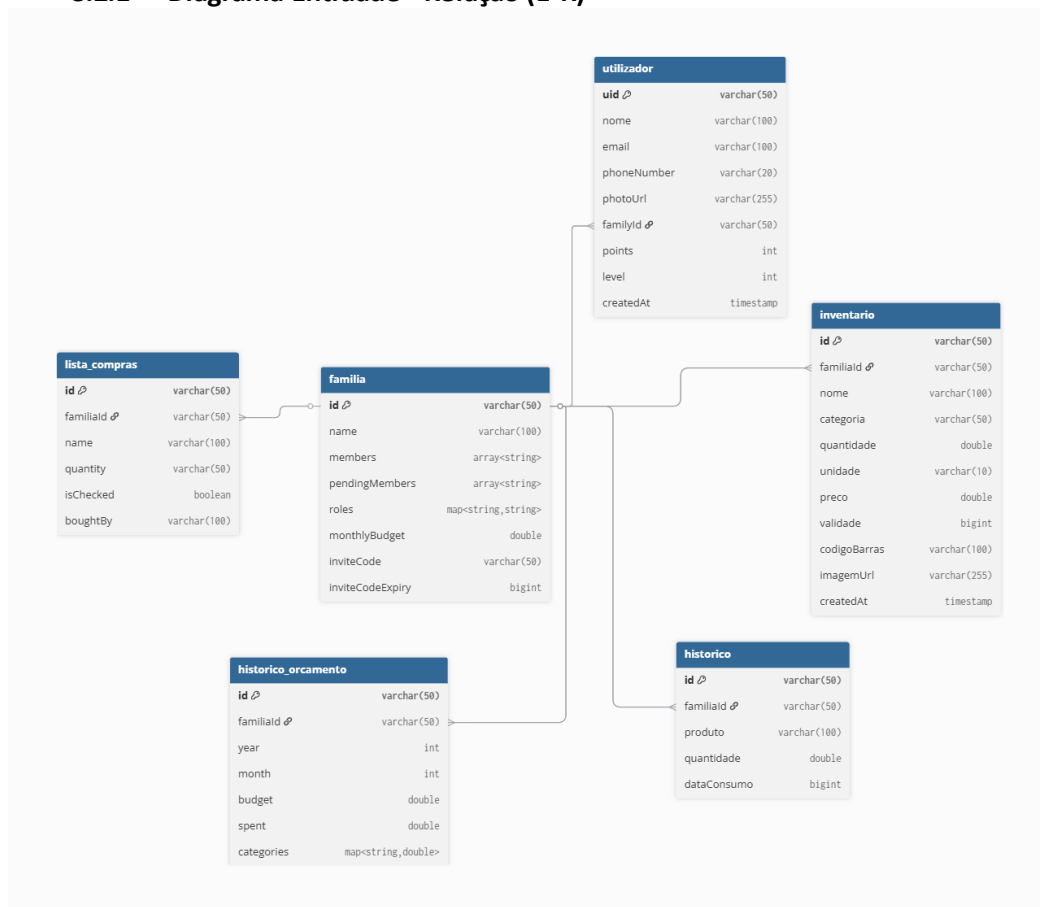


Figura 11 - Diagrama Entidade-Relação.

Relações principais

- **1 Família → N Utilizadores**
Uma família pode ter vários utilizadores (membros), mas cada utilizador pertence apenas a **uma família**.
- **1 Família → N Produtos (Inventário)**
O inventário de produtos é específico de cada família. Um produto pertence a **uma família**.
- **1 Família → N Listas**
Cada família pode criar várias listas de compras. Todos os membros da família partilham o acesso às mesmas listas.
- **1 Lista → N Itens de Lista**
Cada lista contém um conjunto de itens (produtos a comprar).

3.2.2 Modelo de Classes (UML)

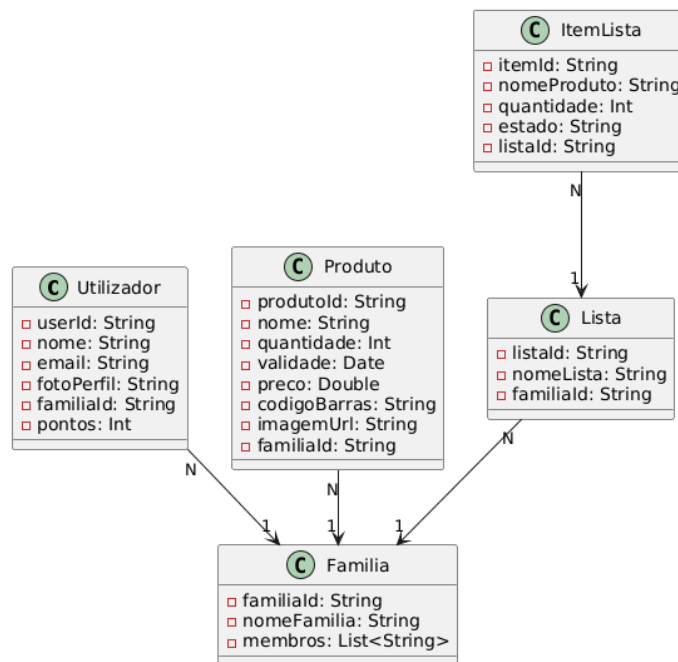


Figura 12 - Modelo de Classes.

3.2.3 Diagramas de Atividade (Activity Diagram)

Os **diagramas de atividade** representam o comportamento dinâmico da aplicação, descrevendo o **fluxo de ações** e decisões que ocorrem durante cada caso de uso.

Estes diagramas permitem compreender a sequência de operações, as interações entre o utilizador e o sistema, e a forma como os dados fluem entre os componentes da aplicação.

Na aplicação **ReStock - Aplicação Inteligente de Gestão de Compras e Inventário Doméstico**, os diagramas de atividade ajudam a ilustrar os principais processos associados aos casos de uso definidos anteriormente.

Cada diagrama mostra de forma clara o papel do utilizador, o processamento realizado pela aplicação e as condições que determinam diferentes fluxos de execução.

O desenvolvimento da aplicação em **Kotlin**, com integração do **Firebase**, influenciou diretamente estes fluxos de atividade, uma vez que muitos processos, como autenticação, leitura de dados e sincronização são automáticos e em tempo real.

Os diagramas seguintes apresentam o funcionamento interno de cada caso de uso principal:

1. **Autenticação de Utilizador:** descreve o processo de validação de credenciais no Firebase Auth e o carregamento dos dados no Firestore.
2. **Adicionar Produto:** mostra o fluxo completo da adição de produtos ao inventário, seja manualmente, por código de barras ou OCR.
3. **Receber Notificações de Validade:** representa a análise periódica das datas de validade e o envio de notificações locais ou push.
4. **Gerir Listas de Compras:** descreve a criação e atualização de listas partilhadas entre membros da família.
5. **Sincronização Familiar:** ilustra o mecanismo de sincronização automática dos dados entre todos os utilizadores da mesma família.

Cada um destes diagramas foi elaborado com base nas funcionalidades planeadas para o **Projeto I**, representando a lógica de funcionamento esperada para cada caso de uso.

Estes modelos servirão também como referência para a fase de **implementação prática no Projeto II**, garantindo a coerência entre o design e o código desenvolvido.

1. Autenticação de Utilizador

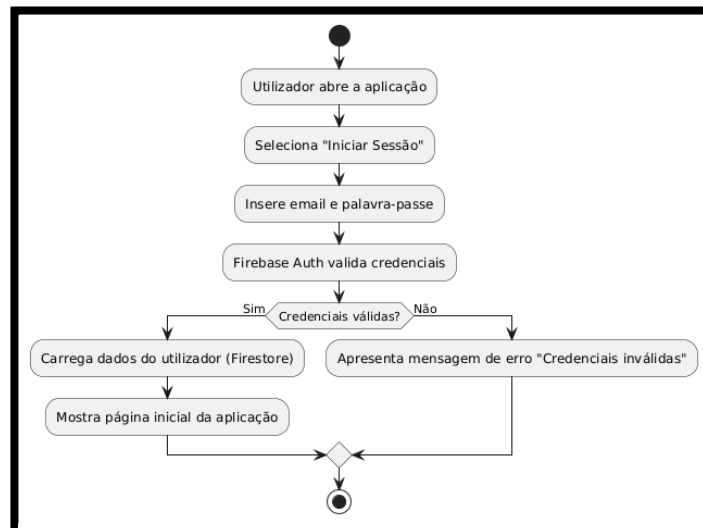


Figura 13 - Diagrama de Atividade Autenticação de Utilizador.

2. Adicionar Produto

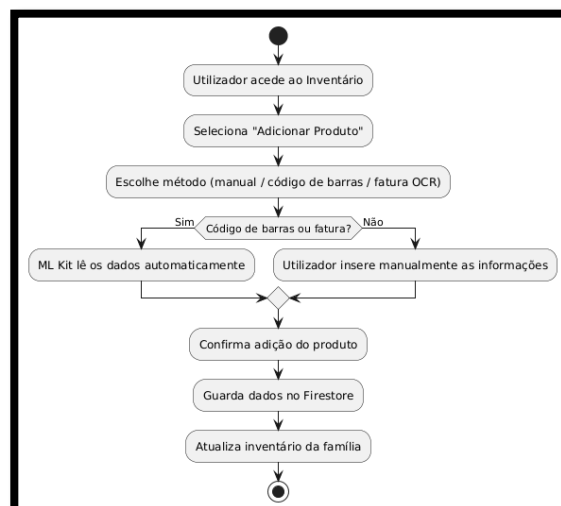


Figura 14 - Diagrama de Atividade Adicionar Produto.

3. Receber Notificações de Validade

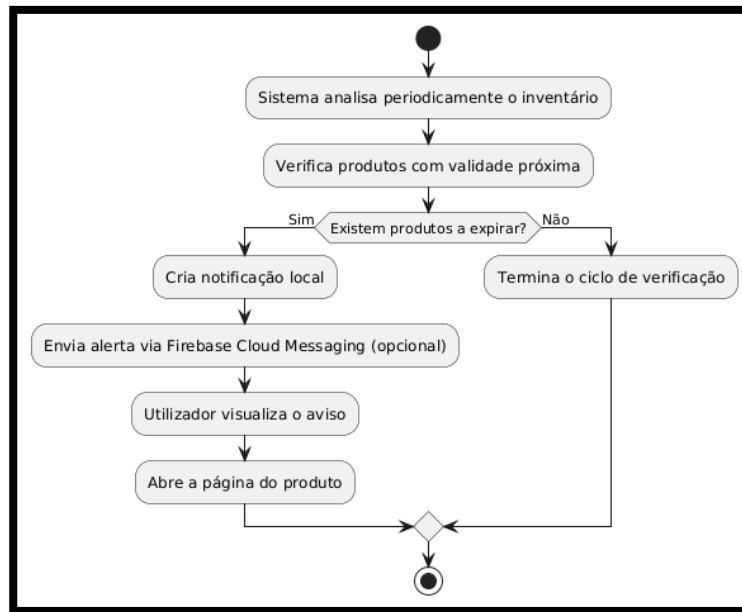


Figura 15 - Diagrama de Atividade Receber Notificações de Validade.

4. Gerir Listas de Compras

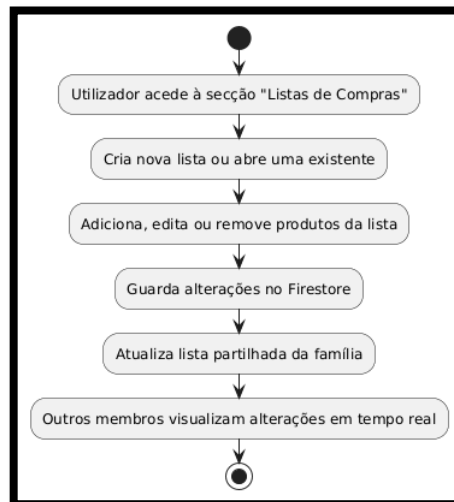


Figura 16 - Diagrama de Atividade Gerir Listas de Compras.

5. Sincronização Familiar

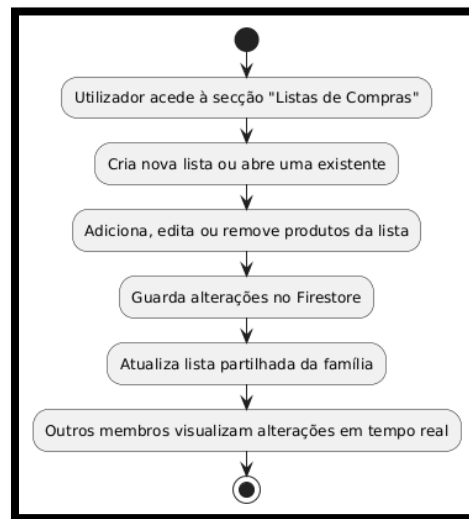


Figura 17 - Diagrama de Atividade
Sincronização Familiar.

3.3 Protótipos de Interface

Com o objetivo de representar visualmente a estrutura e o funcionamento da aplicação **ReStock**, foi desenvolvido um **protótipo de interface** utilizando a ferramenta **Figma**.

Inicialmente foram elaborados protótipos de baixa fidelidade em papel.

O mockup criado reflete a identidade visual da aplicação, seguindo as normas do **Material Design 3**, e demonstra o **fluxo de navegação entre os principais ecrãs**: home, inventário, listas de compras e perfil do utilizador.

Cada ecrã foi projetado com foco na **usabilidade**, **simplicidade** e **coerência visual**, garantindo que o utilizador consiga aceder facilmente a todas as funcionalidades sem necessidade de instruções complexas.

O protótipo inclui os elementos principais da aplicação, como:

- Ecrã **Principal**, com barra de navegação inferior;
- Páginas de **Inventário**, **Listas de Compras**, e **Perfil**, integradas visualmente de forma coerente;
- Cores suaves e ícones personalizáveis, de acordo com a identidade visual da aplicação ReStock.

O protótipo pode ser consultado através do seguinte link:

 [Protótipo ReStock - Figma](#)

Este protótipo permitiu validar o **layout da interface**, o **posicionamento dos componentes** e a **navegação entre ecrãs**, servindo de base para a fase de desenvolvimento prático no **Projeto II**. Com o Figma, foi possível criar uma visão clara e interativa da aplicação final, demonstrando o design funcional e estético planeado para o ReStock.

4 Solução Desenvolvida

4.1 Introdução

A solução desenvolvida, denominada ReStock, é uma aplicação móvel para a plataforma Android concebida para otimizar a gestão de inventários domésticos e a organização de compras. Diferenciando-se das soluções genéricas de mercado, o ReStock oferece um ecossistema integrado que combina a monitorização rigorosa de validade (com alertas visuais e gráficos), controlo orçamental em tempo real e uma componente de gamificação para incentivar o uso responsável e a redução do desperdício alimentar. A aplicação utiliza tecnologias modernas como a leitura de códigos de barras para entrada rápida de dados e um sistema de sugestões inteligente que antecipa necessidades de reposição com base no histórico de consumo e na proximidade das datas de expiração. Além disso, a vertente de gestão familiar permite a sincronização multiutilizador, garantindo que o inventário e a lista de compras estejam sempre atualizados para todos os membros do agregado.

- [ReStock Vídeo](#)
- [GitHub ReStock](#)
- Credenciais de acesso para teste:
 - **Utilizador:** teste.restock2026@gmail.com
 - **Password:** restock2026!
 - **Indicações úteis:** A aplicação requer ligação à internet para sincronização com a base de dados Firebase. Para testar o scanner, pode utilizar qualquer código de barras EAN-13 comum.

Este capítulo descreve detalhadamente os aspetos técnicos e funcionais que compõem o ReStock. Primeiramente, é apresentada a arquitetura do sistema e a stack tecnológica utilizada. De seguida, detalham-se as funcionalidades de core da aplicação, o modelo de dados e a estrutura da base de dados. Por fim, exploram-se as opções de design de interface (UI/UX) e as estratégias implementadas para garantir a segurança e persistência da informação.

4.2 Arquitetura

A arquitetura do ReStock foi desenhada seguindo as recomendações oficiais da Google para o desenvolvimento Android moderno, adotando o padrão MVVM (Model-View-ViewModel). Esta escolha fundamenta-se na necessidade de criar uma aplicação robusta, testável e de fácil manutenção, garantindo uma separação clara entre a interface de utilizador, a lógica de negócio e a persistência de dados.

Componentes da Arquitetura:

1. Camada de Visualização (View):

Composta por Activities e Fragments (ex: HomeFragment, InventoryFragment). Esta camada é estritamente responsável pela renderização da UI e pela interação direta com o utilizador, utilizando View Binding para aceder aos componentes de layout de forma segura.

2. Camada de ViewModel:

Atua como intermediária entre a View e a camada de dados. Os ViewModels (ex: InventarioViewModel, BudgetViewModel) expõem o estado da aplicação através de Kotlin Flows e StateFlow, permitindo que a UI reaja a mudanças nos dados de forma assíncrona e sobreviva a mudanças de configuração.

3. Camada de Repositório (Repository):

Funciona como uma única fonte de verdade (Single Source of Truth). Os repositórios (ex: UserRepository, GamificationRepository) abstraem a origem dos dados, decidindo se a informação deve ser lida da base de dados local ou sincronizada com a nuvem.

4. Camada de Modelo (Model):

Define as entidades de dados do domínio (ex: Product, User, ShoppingListItem), partilhadas por todas as camadas.

5. Serviços Externos (Firebase):

O ReStock utiliza o Firebase Authentication para gestão de identidade e o Cloud Firestore para garantir a sincronização em tempo real entre diferentes membros de um mesmo agregado familiar.

Fundamentação das Opções Arquiteturais:

- **Separação de Preocupações (SoC):** Ao isolar a lógica de negócio (ViewModels) da lógica de apresentação (Fragments), reduz-se a complexidade das classes e facilita-se a introdução de novas funcionalidades sem afetar componentes estáveis.
- **Programação Reativa:** O uso de Coroutines e Flow permite gerir operações pesadas (como chamadas de rede ou I/O) de forma não bloqueante, garantindo que a interface se mantenha fluida e responsiva mesmo durante a sincronização de grandes inventários.
- **Escalabilidade e Sincronização:** A integração com o ecossistema Firebase foi escolhida pela sua capacidade nativa de sincronização offline-first e notificações push, essenciais para uma aplicação de gestão familiar onde múltiplos utilizadores podem alterar a lista de compras simultaneamente.

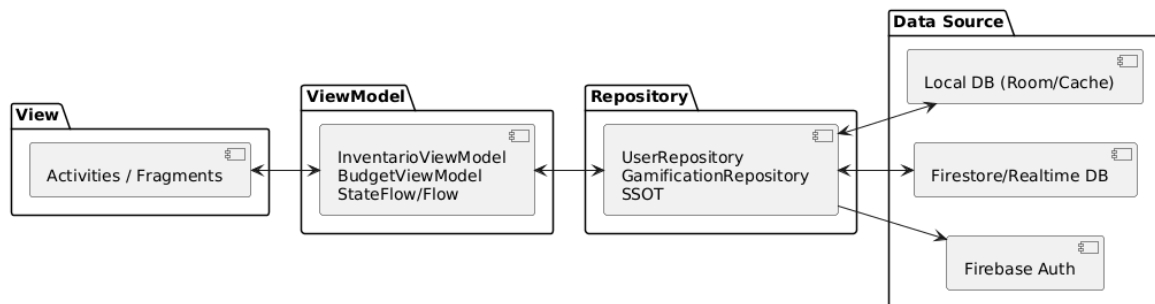


Figura 18 - Diagrama de Componentes Arquitetura MVVM.

4.3 Tecnologias e Ferramentas Utilizadas

A construção da solução ReStock baseia-se num conjunto de tecnologias modernas que asseguram a interoperabilidade, escalabilidade e uma experiência de utilizador fluida. Abaixo, descrevem-se as principais tecnologias e as respetivas fundamentações.

Tecnologias:

- **Kotlin (Linguagem de Programação):** Utilizada como linguagem principal devido à sua concisão, segurança de tipos (especialmente o tratamento de nulls) e suporte nativo para Coroutines, essencial para operações assíncronas.
- **Android SDK (API Level 29+):** Escolhida para garantir compatibilidade com dispositivos modernos, permitindo o uso de funcionalidades avançadas de hardware e segurança.
- **Firebase (BaaS):**
 - **Cloud Firestore:** Base de dados NoSQL orientada a documentos para armazenamento e sincronização em tempo real. Escolhida para suportar a funcionalidade de gestão familiar em múltiplos dispositivos.
 - **Firebase Authentication:** Para gestão segura de identidades (email/password e Google Sign-In), com suporte a Autenticação Multi-Fator (MFA) via SMS (implementado em MfaActivity.kt), reforçando a segurança das contas em conformidade com o RNF07.
 - **Firebase Storage:** Para armazenamento de imagens de perfil e produtos.

Bibliotecas e Frameworks:

- **Jetpack Navigation Component:** Implementado para gerir o fluxo de navegação entre ecrãs (Fragments) de forma centralizada e segura (Safe Args), facilitando a transição entre Inventário, Lista de Compras e Orçamento.
- **CameraX & ML Kit Barcode Scanning:** Utilizados para a funcionalidade de scanner de códigos de barras, permitindo uma entrada de dados rápida e precisa, minimizando erros manuais do utilizador.
- **MPAndroidChart:** Biblioteca para visualização de dados, usada para gerar o gráfico circular de estado do inventário (produtos válidos vs. expirados) e dashboards de orçamento.
- **Glide:** Utilizada para o carregamento eficiente e caching de imagens, garantindo que a visualização de produtos e perfis não comprometa a performance da UI.
- **WorkManager:** Para agendamento de tarefas em segundo plano, como a verificação periódica de validades e envio de notificações automáticas.

Ferramentas de Desenvolvimento

Ferramenta	Descrição e Justificação
Android Studio (Ladybug)	IDE oficial de desenvolvimento, utilizada para codificação, depuração e profiling da aplicação.
Gradle	Sistema de automação de build que gere dependências e configurações de compilação de forma eficiente.
GitHub	Sistema de controlo de versões para gestão do código fonte e colaboração, garantindo a rastreabilidade das alterações.
Figma	Ferramenta de design utilizada para a prototipagem da interface e definição da experiência de utilizador (UI/UX).

Tabela 9 - Ferramentas de Desenvolvimento

Fundamentação Tecnológica:

A escolha desta stack tecnológica deve-se ao compromisso entre rapidez de desenvolvimento (através do Firebase e bibliotecas Jetpack) e qualidade da aplicação final. A tecnologia offline-first do Firestore garante que o utilizador possa consultar a sua lista de compras mesmo dentro de um supermercado com má cobertura de rede, sincronizando os dados assim que a ligação é restabelecida. O uso do ML Kit para leitura de códigos de barras responde diretamente ao requisito de usabilidade, tornando a tarefa de adicionar produtos ao inventário menos morosa.

ReStock — Arquitetura com Perímetros Tecnológicos

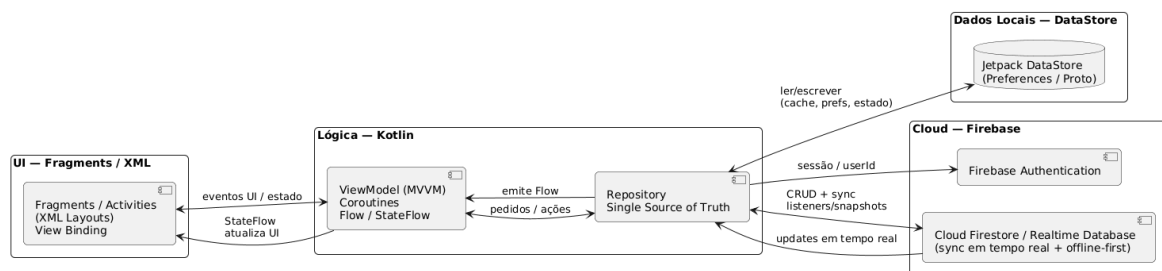


Figura 19 - Diagrama de Arquitetura em Camadas.

4.4 Ambientes de Teste e de Produção

O ambiente de produção da solução ReStock assenta exclusivamente em serviços cloud, pelo que não requer infraestrutura física dedicada por parte do utilizador final.

- **Requisitos mínimos do dispositivo cliente:** processador de 1,4 GHz dual-core, 2 GB de RAM, 100 MB de armazenamento interno livre e ligação à internet (Wi-Fi ou dados móveis de 4G). O sistema operativo mínimo suportado é o Android 10 (API Level 29).

No que diz respeito aos recursos cloud, a solução utiliza os serviços Firebase no plano Blaze (pay-as-you-go). Este plano inclui uma camada gratuita equivalente ao Spark Plan nomeadamente 1 GB de armazenamento no Cloud Firestore, 50 000 leituras diárias gratuitas e 5 GB de armazenamento no Firebase Storage, permitindo adicionalmente escalar os recursos de forma automática e sem limitações fixas caso o volume de utilização o exija. O Firebase Authentication não impõe limite de utilizadores ativos em nenhum dos planos. O plano Blaze foi adotado por ser requisito para a utilização de determinadas funcionalidades avançadas da plataforma, como o Firebase App Check e a integração com Cloud Functions, garantindo assim maior flexibilidade para evoluções futuras da solução.

O ambiente de teste utilizado durante o desenvolvimento incluiu o emulador do Android Studio (Medium Phone API 36.1) e um tablet físico Lenovo Tab M10. Os testes de sincronização foram realizados com o emulador e o tablet a correr em simultâneo, cada um autenticado com uma conta de utilizador distinta, validando a atualização em tempo real dos dados através do Firebase Firestore.

4.5 Abrangência

Na solução proposta foram aplicadas várias **unidades curriculares** e **áreas científicas** do curso, sobretudo nas componentes de desenvolvimento, dados, redes e engenharia de software.

- **Programação / Programação Orientada a Objetos:** utilizada na implementação da aplicação em Kotlin, na definição de classes, modelos de dados e lógica funcional.
- **Bases de Dados:** aplicada na modelação e organização da informação de utilizadores, famílias, produtos e listas, recorrendo ao Firestore.
- **Engenharia de Software:** usada na estruturação da solução, na definição da arquitetura, organização por camadas e adoção de boas práticas de desenvolvimento e manutenção.
- **Sistemas Distribuídos:** aplicada no uso de serviços cloud para autenticação, armazenamento e sincronização de dados em tempo real entre vários utilizadores.
- **Redes de Computadores:** utilizada na comunicação entre a aplicação móvel, os serviços Firebase e APIs externas.
- **Desenvolvimento de Aplicações Móveis:** aplicada na construção da interface Android, navegação entre ecrãs, usabilidade e interação com o utilizador.
- **Engenharia de Requisitos e Análise de Sistemas:** usada no levantamento das necessidades do problema, definição das funcionalidades e adequação da solução ao contexto de uso.

4.6 Componentes

4.6.1 Componente 1: Aplicação móvel Android

Este componente corresponde à aplicação cliente da solução, desenvolvida para dispositivos Android, sendo responsável pela interação direta com o utilizador. A sua implementação foi realizada em Kotlin, com recurso a uma organização modular baseada em **Activities**, **Fragments**, **ViewModels** e **Repositories**, permitindo separar adequadamente a interface, a lógica da aplicação e a camada de acesso aos dados.

Neste componente encontram-se implementadas as principais funcionalidades da solução, nomeadamente a autenticação de utilizadores, a gestão de inventário, a lista de compras, a gestão familiar, o controlo de orçamento, a gamificação, as definições e a área de conta do utilizador. A navegação entre ecrãs é assegurada pelo **Navigation Component**, enquanto a construção da interface gráfica recorre a **View Binding** e a componentes **Material Design**, contribuindo para uma estrutura organizada, segura e coerente do ponto de vista da usabilidade.

4.6.2 Componente 2: Backend e persistência de dados em cloud

Este componente é responsável pela gestão e persistência remota dos dados da solução, bem como pela sincronização entre utilizadores e dispositivos. A sua implementação assenta na plataforma Firebase, através dos serviços **Firebase Authentication**, **Cloud Firestore** e **Firebase Storage**.

O Firebase Authentication é utilizado para suportar o registo e autenticação de utilizadores, incluindo autenticação por **email e palavra-passe** e por **Google Sign-In**. O **Cloud Firestore** é utilizado como base de dados principal, armazenando informação relativa a utilizadores, famílias, produtos, listas de compras, histórico e restantes dados funcionais da aplicação. O **Firebase Storage** é utilizado para armazenar ficheiros, nomeadamente imagens de perfil e imagens associadas aos produtos. Este componente permite suportar uma utilização colaborativa e sincronizada em tempo real, sem necessidade de infraestrutura de servidor própria.

4.6.3 Componente 3: Leitura de código de barras e obtenção automática de dados

Este componente tem como objetivo apoiar e automatizar o registo de produtos no inventário. A sua implementação recorre à biblioteca **CameraX**, para acesso à câmara do dispositivo, e ao **ML Kit Barcode Scanning**, para deteção e leitura de códigos de barras em tempo real.

Após a leitura do código, a aplicação pode consultar serviços externos, como **OpenFoodFacts**, **OpenProductsFacts** e **OpenBeautyFacts**, de forma a obter automaticamente informação relevante sobre o produto, como designação, imagem e categoria. Esta componente reduz a introdução manual de dados, melhora a eficiência do processo de registo e aumenta a comodidade de utilização da solução.

4.6.4 Componente 4: Notificações e gestão local de preferências

Este componente é responsável pelas funcionalidades locais complementares da aplicação, relacionadas com personalização e execução de tarefas em segundo plano. A gestão de preferências locais, como idioma, tema e ativação de notificações, é implementada através de **DataStore**, permitindo a persistência destes dados no dispositivo de forma segura e eficiente.

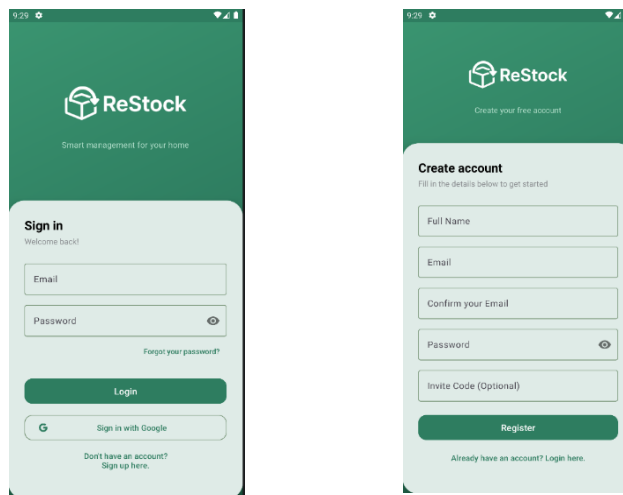
Adicionalmente, este componente integra o **WorkManager**, utilizado para a execução periódica de tarefas em segundo plano, nomeadamente a verificação de produtos próximos do fim da validade e a emissão de notificações locais ao utilizador. Esta abordagem permite que a aplicação disponibilize mecanismos de alerta e apoio à utilização contínua, mesmo quando não se encontra aberta em primeiro plano.

4.7 Interfaces

Nesta secção apresenta-se o mapa aplicacional da solução, através dos ecrãs mais representativos da aplicação. Cada interface foi desenvolvida com o objetivo de garantir simplicidade de utilização, organização visual e facilidade de navegação, tendo sido implementada com recurso a **layouts XML, Fragments/Activities, View Binding e Navigation Component**.

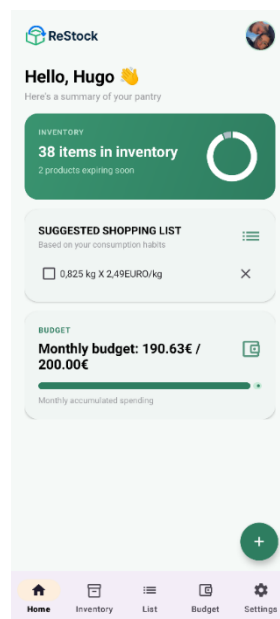
Ecrã de autenticação

Este ecrã permite ao utilizador iniciar sessão na aplicação, através de **email e palavra-passe** ou de **Google Sign-In**. A sua implementação foi realizada numa **Activity** dedicada, ligada ao **Firestore Authentication** para validação das credenciais. Na sua conceção optou-se por uma interface simples e direta, com os elementos essenciais de autenticação e acesso ao registo, de forma a reduzir a complexidade na entrada inicial na aplicação.



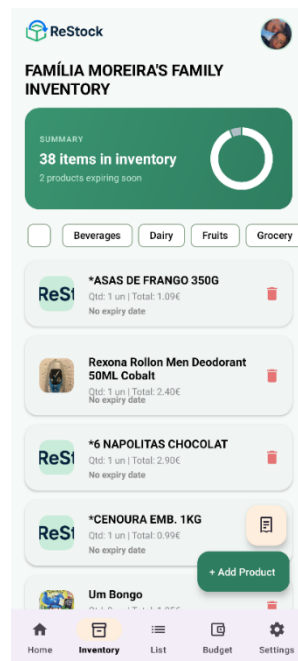
Ecrã principal / Home

Este ecrã funciona como ponto central da aplicação após autenticação, permitindo o acesso às várias áreas funcionais através da barra de navegação inferior. A implementação foi feita numa **Activity** principal com um **NavHostFragment**, responsável por alojar os diferentes fragmentos da aplicação. A principal decisão de implementação passou por centralizar a navegação neste ecrã, garantindo consistência e melhor organização da experiência do utilizador.



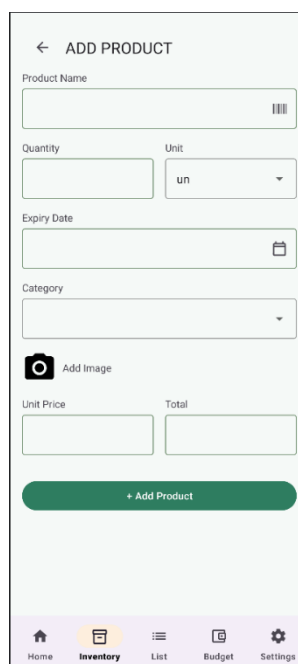
Ecrã de inventário

Este ecrã permite visualizar os produtos armazenados, consultar informações relevantes e gerir o stock existente. A sua implementação foi feita com um **Fragment**, associado a um **ViewModel**, permitindo observar e atualizar dados provenientes do **Firestore**. Foi tomada a decisão de apresentar a informação de forma clara e listada, facilitando a consulta rápida do inventário e a identificação de produtos.



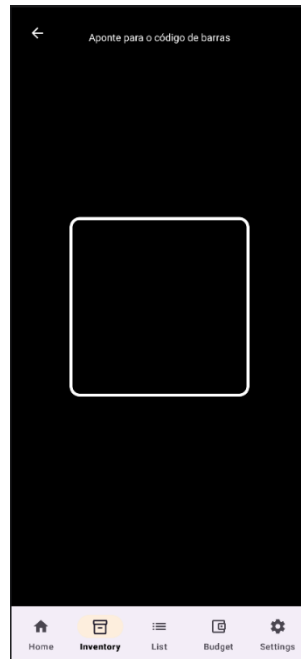
Ecrã de adição/edição de produto

Este ecrã é utilizado para registar novos produtos ou editar produtos já existentes. Inclui campos como nome, categoria, quantidade, preço, validade e imagem. A interface foi implementada num **Fragment** com formulários em XML, lógica de validação local e integração com **Firestore Storage** para upload de imagens. Foi também integrada a leitura de código de barras e o preenchimento automático de dados, como forma de reduzir o esforço manual do utilizador.



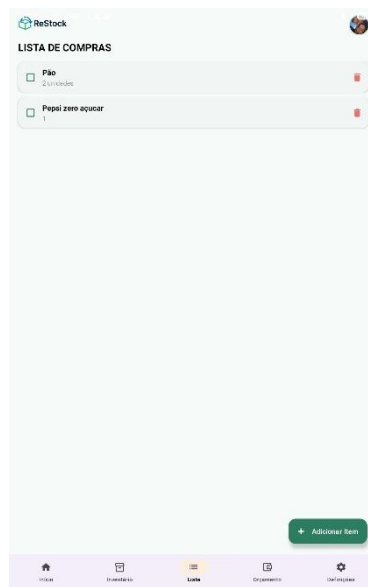
Ecrã de leitura de código de barras

Este ecrã permite captar códigos de barras através da câmara do dispositivo. A sua implementação recorre a **CameraX** para pré-visualização da imagem e a **ML Kit** para deteção e leitura do código. A principal decisão tomada foi automatizar o regresso ao ecrã de produto após deteção de um código válido, tornando o processo mais rápido e intuitivo.



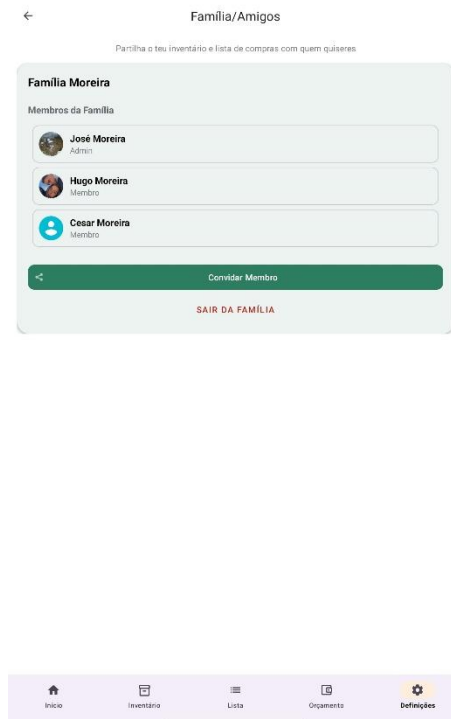
Ecrã da lista de compras

Este ecrã apresenta os itens em falta ou necessários para compra, funcionando como complemento ao inventário. Foi implementado com um **Fragment** e um **ViewModel**, com dados sincronizados em tempo real a partir do **Firestore**. Optou-se por uma organização simples, centrada na visualização rápida dos produtos e no apoio à gestão doméstica.



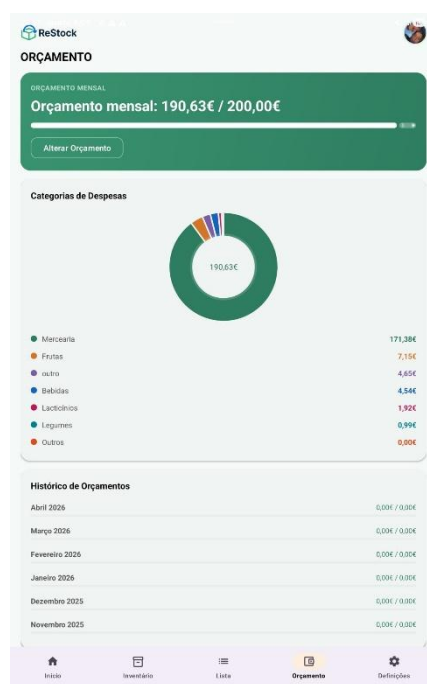
Ecrã de família

Este ecrã permite visualizar os membros do grupo, gerar códigos de convite e gerir aspetos associados à colaboração entre utilizadores. A sua implementação foi feita com ligação ao **Cloud Firestore**, permitindo manter os dados sincronizados entre dispositivos. A decisão principal foi suportar um modelo colaborativo simples, adequado ao contexto familiar da aplicação.



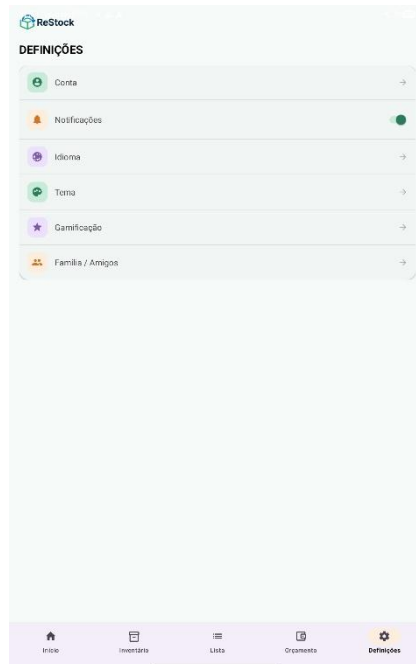
Ecrã de orçamento

Este ecrã apresenta informação financeira associada aos produtos e gastos da família. A implementação foi feita com um **Fragment** e apoio da biblioteca **MPAndroidChart** para representação gráfica. A opção por gráficos e indicadores visuais teve como objetivo facilitar a interpretação da informação e apoiar o controlo de despesas.



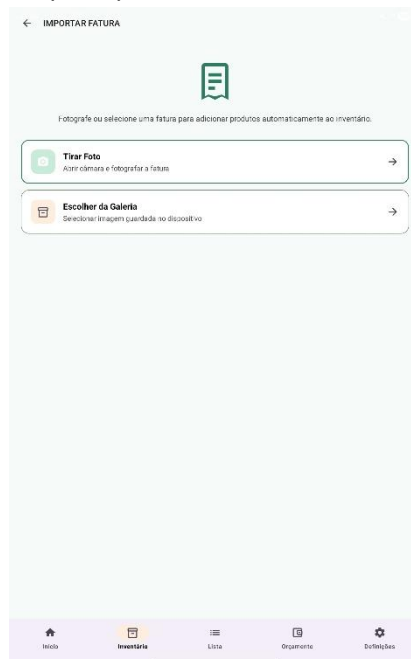
Ecrã de conta e definições

Este conjunto de ecrãs permite ao utilizador consultar e alterar dados pessoais, imagem de perfil, idioma, tema e notificações. A implementação recorre a **Firestore Authentication**, **Firestore Storage** e **DataStore**. Foi tomada a decisão de separar as preferências locais das informações de conta, de forma a distinguir claramente a personalização da aplicação da gestão do perfil do utilizador.



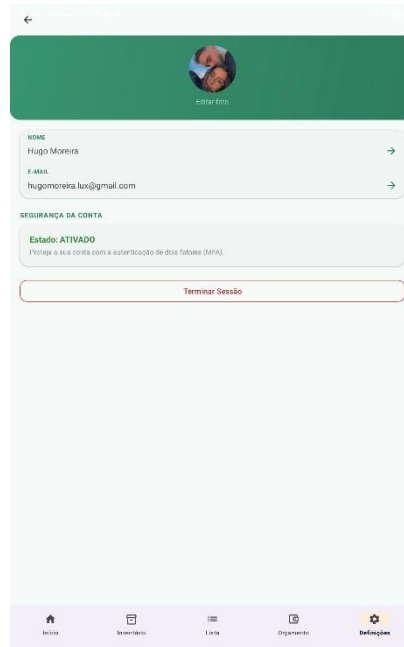
Ecrã de digitalização de fatura (OCR)

Este ecrã permite ao utilizador fotografar uma fatura de supermercado ou selecionar uma imagem da galeria, processando-a com o ML Kit Text Recognition (OCR) para extrair automaticamente os produtos identificados. Após o processamento, é apresentado um ecrã de revisão (InvoiceReviewFragment) onde o utilizador confirma, edita ou remove os itens detetados antes de os adicionar ao inventário. Esta funcionalidade reduz significativamente o esforço manual de registo de compras, sendo um dos principais diferenciadores face às soluções concorrentes.



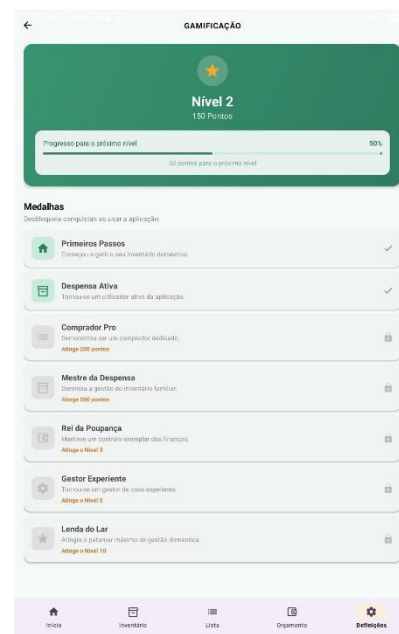
Ecrã de segurança e autenticação multi-fator (MFA)

Este ecrã é dedicado à gestão da segurança da conta do utilizador, nomeadamente a ativação e gestão da Autenticação Multi-Fator (MFA) através de número de telefone. Implementado na *SecurityFragment* e *MfaActivity*, o fluxo envia um código OTP por SMS via *Firebase Authentication*, exigindo a sua validação para completar o acesso. Esta funcionalidade reforça a proteção da conta, sendo especialmente relevante em contextos de partilha familiar.



Gamificação

Este ecrã apresenta ao utilizador o seu progresso no sistema de pontos e níveis da aplicação, funcionando como um mecanismo de motivação para a adoção de boas práticas de consumo e redução do desperdício alimentar. A sua implementação foi realizada num *Fragment* dedicado (*GamificationFragment*), associado a um *GamificationViewModel* que observa reativamente os dados do utilizador no *Firestore* através de um *snapshot listener*, atualizando a interface em tempo real. O ecrã exibe o nível atual e o total de pontos acumulados, acompanhados de uma barra de progresso que indica a percentagem de avanço para o nível seguinte, calculada com base na fórmula $\text{pontos} \% 100$. Complementarmente, é apresentada uma secção de *badges*, composta por sete conquistas desbloqueáveis mediante a atingir marcos de pontos ou níveis (por exemplo, "First Steps" aos 10 pontos ou "Home Legend" ao nível 10), com representação visual distinta entre *badges* bloqueados e desbloqueados. A principal decisão de design passou por tornar o progresso transparente e imediato para o utilizador, recorrendo a componentes *Material Design 3* e a um *BadgeAdapter* com *DiffUtil* para garantir atualizações eficientes da lista.



5 Testes e Validação

A fase de testes e validação do ReStock foi desenhada não apenas para garantir a ausência de erros técnicos (bugs), mas para certificar que a solução cumpre o seu propósito fundamental: a gestão eficiente de recursos alimentares e a redução do desperdício em ambiente doméstico. A estratégia de validação seguiu uma abordagem multicamada, cobrindo desde a integridade do código até à operação em ambiente real com serviços cloud.

5.1 Abordagem e Justificação

Para validar a solução, utilizou-se a pirâmide de testes de software, garantindo que cada componente (desde a lógica de dados até à interface) fosse verificado. A escolha de testes automatizados (JUnit e Espresso) justifica-se pela necessidade de garantir a fiabilidade das regras de negócio (gamificação e orçamentos), enquanto o teste de stress (ADB Monkey) validou a robustez da aplicação perante interações imprevistas.

5.2 Análise de Risco e Impacto

Recorreu-se a um modelo formal de análise de riscos para identificar pontos críticos na operação da solução em contexto produtivo:

Tabela 10 - Análise de Risco e Impacto

Risco Identificado	Probabilidade	Impacto	Estratégia de Mitigação
Falha de Conetividade Cloud	Média	Elevado	Implementação de persistência offline via Firestore (cache local).
Acesso Indevido a Dados	Baixa	Crítico	Configuração de Firestore Security Rules baseadas em UID e FamilyID.
Corrupção de Inventário	Baixa	Médio	Utilização de Write Batches e Transactions para atomicidade de dados.
Exaustão de Recursos (Memory Leak)	Baixa	Médio	Monitorização em tempo real com o Android Profiler.

5.3 Validação de Qualidade Técnica e Funcional

A qualidade do software foi validada através de dois tipos de testes automatizados:

- Testes Unitários (Logic Validation):** Foram realizados cinco scripts de testes unitários que validam a integridade e o comportamento dos componentes centrais da aplicação:
 - (1) `ModelIntegrityTest.kt` – valida a integridade dos modelos de dados e a correta atribuição de pontos e níveis no sistema de gamificação;
 - (2) `BudgetCalculationTest.kt` – verifica o cálculo do total gasto no mês atual, os gastos por categoria e o filtro de produtos de meses anteriores;
 - (3) `CategoryUtilsTest.kt` – testa a normalização bidirecional de categorias (PT ↔ EN ↔ chave neutra) e a insensibilidade a maiúsculas;
 - (4) `ProductModelTest.kt` – valida a criação de entidades Product e os valores por omissão do modelo de dados;
 - (5) `ExampleUnitTest.kt` – teste de sanidade da plataforma JUnit.
- Testes de Interface (UI/UX Validation):** Utilizou-se a framework Espresso (`LoginUITest.kt`) para validar o fluxo crítico de autenticação. Verificou-se a renderização correta dos componentes de login e a disponibilidade dos serviços de autenticação do Google, garantindo que a porta de entrada da aplicação é resiliente.

Os testes de validação operacional em contexto real, realizados com um grupo familiar, demonstraram que a sincronização entre dispositivos ocorre em tempo real, as notificações de validade são emitidas corretamente e a lista de compras partilhada funciona sem conflitos entre utilizadores.

Os detalhes dos test cases e respetivos resultados encontram-se descritos no Anexo de Testes, onde se incluem os cenários testados, os inputs utilizados, os resultados esperados e os resultados obtidos para cada caso de uso principal.

5.4 Validação Operacional e Cloud

A validação operacional incidiu sobre os recursos indicados e a sua performance em ambiente real:

- Recursos Cloud (Firebase): Validou-se a latência de sincronização entre múltiplos dispositivos familiares. Em testes de rede Wi-Fi, as atualizações da lista de compras refletiram-se em todos os terminais em poucos segundos.
- Armazenamento e Rede: O upload de imagens de produtos para o Firebase Storage foi testado para garantir que a compressão de imagem não compromete a experiência de rede nem esgota o armazenamento do dispositivo.
- Monitorização de Recursos: Através do Android Profiler, confirmou-se que a aplicação mantém um consumo estável de CPU e memória, mesmo sob o stress de 1000 eventos aleatórios gerados pela ferramenta ADB Monkey.

5.5 Validação em Contexto Real e Participação de Terceiros

A pertinência da solução foi testada em ambiente real através de um grupo de controlo composto por membros de um agregado familiar (terceiros).

- Cenário de Teste: Um agregado familiar de quatro pessoas utilizou a aplicação durante uma semana, registando os produtos adquiridos semanalmente, consultando o inventário antes de ir às compras e gerindo a lista de compras partilhada entre dois dispositivos distintos.
- Resultados Observados: A sincronização entre dispositivos funcionou corretamente em todos os cenários testados. As notificações de validade foram emitidas com a antecedência configurada. A lista de compras partilhada refletiu em tempo real as alterações efetuadas por diferentes membros da família. A taxa de sucesso dos testes funcionais foi de 100% para os fluxos principais.
- Feedback Operacional: Os utilizadores consideraram a aplicação intuitiva e útil no contexto doméstico. Foram identificadas sugestões de melhoria, nomeadamente a adição de filtros por categoria no inventário e a possibilidade de definir quantidades mínimas de alerta por produto, que serão consideradas em trabalhos futuros.

5.6 Conclusão da Validação

Os resultados obtidos demonstram que o ReStock é uma solução tecnicamente sólida e operacionalmente relevante. A integração bem-sucedida entre o hardware móvel e os serviços cloud, validada por testes de stress, confirma que a aplicação atinge os seus objetivos de aplicabilidade e utilidade social na luta contra o desperdício alimentar.

6 Método e Planeamento

O desenvolvimento do projeto ReStock seguiu uma abordagem incremental e iterativa, organizada em duas fases principais ao longo do Trabalho Final de Curso.

No âmbito do Projeto I (1.º semestre), o trabalho incidiu essencialmente na componente teórica e de planeamento. Foram definidos o tema, os objetivos e o problema a resolver, seguindo-se o levantamento de requisitos, a análise de soluções existentes e a elaboração dos primeiros protótipos de interface. Posteriormente, foram desenvolvidos os modelos de dados, a arquitetura da solução e os diagramas de atividade, culminando na entrega intercalar de novembro de 2025 e no relatório final do Projeto I em janeiro de 2026.

No âmbito do Projeto II (2.º semestre), o foco transitou para a implementação prática da solução. Foram desenvolvidas todas as funcionalidades da aplicação, incluindo a autenticação, a gestão de inventário, a lista de compras, a partilha familiar, o controlo de orçamento, a gamificação e a digitalização de faturas por OCR. Esta última exigiu mais tempo de investigação do que o inicialmente previsto, tendo sido concluída com sucesso na fase final do semestre.

O planeamento foi acompanhado por um cronograma (Gantt Chart) que permitiu gerir as tarefas e os prazos ao longo de ambos os semestres. As principais fases foram: setembro - outubro 2025 (enquadramento, requisitos e análise); Novembro 2025 (modelação, protótipos e entrega intercalar); Janeiro 2026 (entrega do relatório intercalar do Projeto I); Fevereiro–Abril 2026 (implementação das funcionalidades core e testes); Maio 2026 (validação, testes finais e entrega do relatório final).

O cumprimento do calendário foi globalmente satisfatório, tendo todos os requisitos funcionais prioritários sido implementados dentro do prazo previsto.

Nome	Início	Fim	Antecessores
Definição da ideia e recolha de requisitos iniciais	15/09/25, 08:00	01/10/25, 17:00	
Elaboração e submissão da Autoproposta	24/09/25, 08:00	01/10/25, 17:00	
Aprovação da Autoproposta e início da investigação teórica	02/10/25, 08:00	07/10/25, 17:00	1;2
Primeira versão do Project Charter	08/10/25, 08:00	15/10/25, 17:00	3
Conclusão da pesquisa de benchmarking e análise SWOT	16/10/25, 08:00	27/10/25, 17:00	4
Mockups e estrutura da aplicação	28/10/25, 09:00	07/11/25, 17:00	4;5
Segunda versão do Project Charter (com Mockups, Benchmarking e SWOT)	10/11/25, 08:00	12/11/25, 17:00	4;5;6
Revisão dos requisitos e diagramas UML	13/11/25, 09:00	19/12/25, 17:00	7
Desenvolvimento da última versão do Project Charter	29/12/25, 09:00	02/01/26, 17:00	7;8
Preparação da apresentação final do Project Charter	05/01/26, 08:00	12/01/26, 16:00	9
Revisão e planeamento do desenvolvimento	02/02/26, 09:00	06/02/26, 17:00	9
Configuração do ambiente de desenvolvimento	07/02/26, 09:00	10/02/26, 17:00	
Desenvolvimento do módulo de autenticação e base de dados	11/02/26, 08:00	24/02/26, 16:00	12
Implementação do módulo de inventário e leitura de códigos de barras	26/02/26, 09:00	13/03/26, 17:00	13
Desenvolvimento do módulo de listas de compras e alertas de validade	16/03/26, 09:00	03/04/26, 17:00	14
Implementação do módulo de receitas e sugestões inteligentes	06/04/26, 08:00	20/04/26, 17:00	15
Desenvolvimento do sistema de gamificação e estatísticas	21/04/26, 08:00	05/05/26, 17:00	16
Testes funcionais e correção de erros	06/05/26, 08:00	20/05/26, 17:00	17
Preparação da apresentação final do Projeto	21/05/26, 08:00	29/05/26, 17:00	18
Entrega e Apresentação Final do Projeto II	30/05/26, 08:00	01/06/26, 17:00	19

Figura 20 Milestones

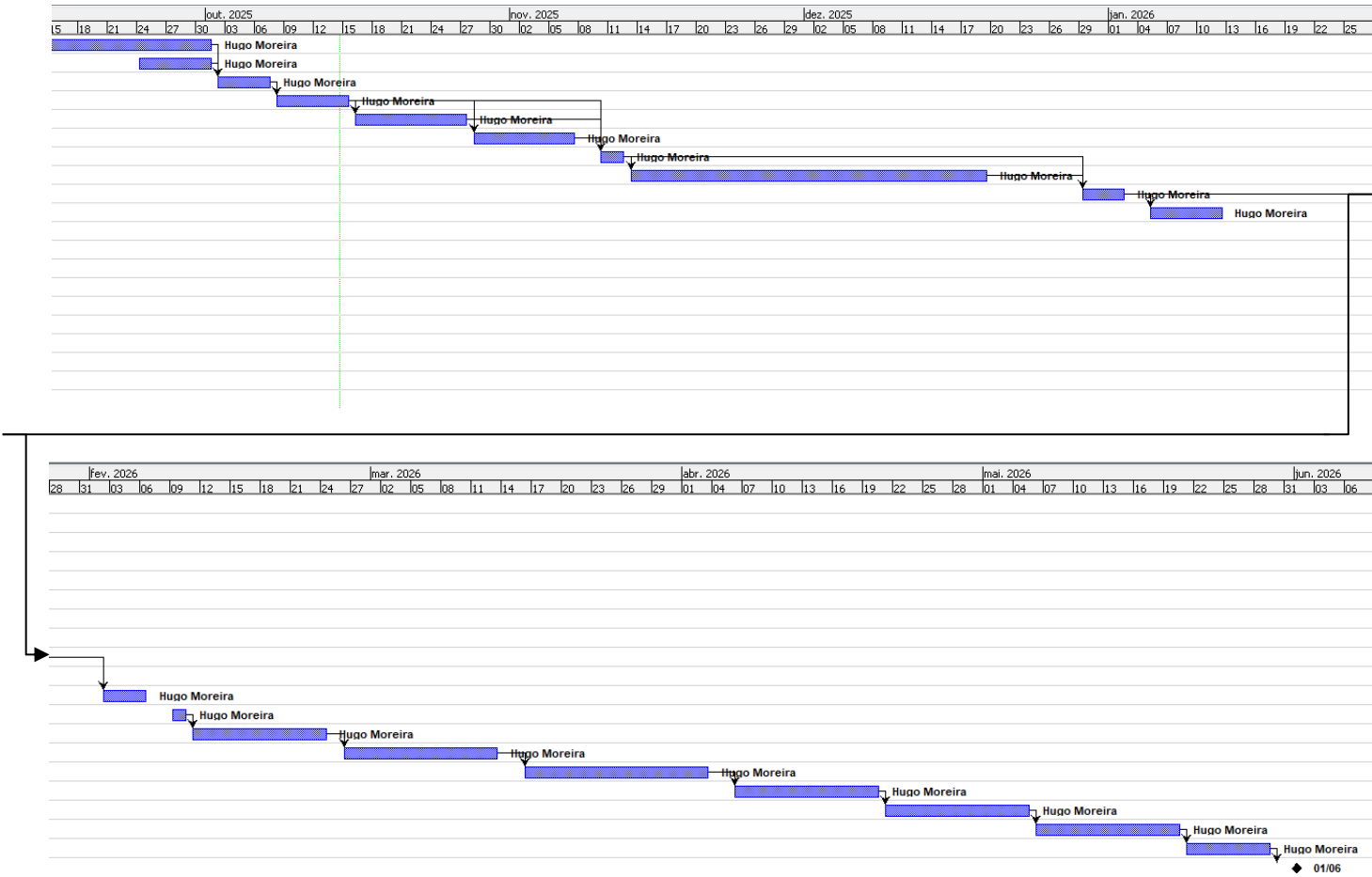


Figura 21 Diagrama de Gantt

7 Resultados

7.1 Resultados dos testes

Os resultados dos testes realizados ao longo do projeto demonstram que a solução ReStock cumpre os principais objetivos técnicos e funcionais definidos.

Os testes unitários realizados validaram a integridade dos modelos de dados e a correta atribuição de pontos no sistema de gamificação (ModelIntegrityTest.kt), os cálculos de orçamento mensal por categoria (BudgetCalculationTest.kt), a normalização de categorias em múltiplos idiomas (CategoryUtilsTest.kt) e a criação correta de entidades de produto (ProductModelTest.kt). Os testes de interface com a framework Espresso (LoginUITest.kt) confirmaram o funcionamento correto do fluxo de autenticação.

Os testes de validação operacional em contexto real, realizados com um grupo familiar, demonstraram que a sincronização entre dispositivos ocorre em tempo real, as notificações de validade são emitidas corretamente e a lista de compras partilhada funciona sem conflitos entre utilizadores.

7.2 Cumprimento de requisitos

A tabela seguinte apresenta o estado de cumprimento de cada um dos requisitos funcionais identificados no Capítulo 3, com indicação do grau de implementação alcançado no âmbito do Projeto I:

Tabela 11 - Estado de cumprimento de Requisitos funcionais

ID	Requisito Funcional	Estado	Observação
RF01	Autenticação de Utilizadores	Realizado	—
RF02	Gestão de Perfil	Realizado	—
RF03	Adicionar Produto Manualmente	Realizado	—
RF04	Leitura de Código de Barras	Realizado	—
RF05	Digitalização de Faturas (OCR)	Realizado	InvoiceScannerFragment + ML Kit implementados
RF06	Atualizar e Remover Produtos	Realizado	—
RF07	Notificações de Validade	Realizado	WorkManager + ExpirationWorker
RF08	Criação Automática de Lista de Compras	Realizado	—
RF09	Gestão Manual da Lista de Compras	Realizado	—
RF10	Partilha Familiar	Realizado	—
RF11	Sincronização em Tempo Real	Realizado	Firebase Firestore
RF12	Recomendações de Receitas	Não Realizado	Trabalho futuro
RF13	Gamificação	Realizado	Futuramente pode ser melhorado
RF14	Comparação de Preços	Não Realizado	Trabalho Futuro
RF15	Armazenamento de Imagens	Realizado	Firebase Storage

Taxa global de cumprimento: 13 requisitos realizados, e 2 não realizados (RF12 – Receitas; RF14 – Comparação de Preços). Taxa global: ~87% (13/15 concluídos integralmente).

8 Conclusão

Em conclusão, o presente trabalho permitiu desenvolver integralmente a aplicação móvel **ReStock**, desde o planeamento e especificação até à implementação, testes e validação. Ao longo dos dois semestres foram identificados os principais problemas associados à gestão do inventário doméstico, definida a arquitetura da solução e implementadas todas as funcionalidades prioritárias, incluindo a leitura de códigos de barras, a digitalização de faturas por OCR, a partilha familiar em tempo real, o controlo de orçamento e o sistema de gamificação.

O projeto demonstrou que é possível desenvolver uma solução técnica sólida, utilizando exclusivamente ferramentas gratuitas e tecnologias modernas, com impacto real na qualidade de vida dos utilizadores. A validação junto do público-alvo, através de inquérito e testes em contexto real, confirmou a pertinência e viabilidade da solução proposta.

Trabalhos Futuros

No âmbito dos trabalhos futuros, prevê-se a implementação das funcionalidades em falta, nomeadamente o sistema de receitas personalizadas com base nos produtos disponíveis no inventário.

Adicionalmente, será considerada a integração com APIs de supermercados nacionais para sugestão de promoções e comparação de preços. A longo prazo, equaciona-se a publicação da aplicação na Google Play Store e a exploração de um modelo de negócio freemium, com funcionalidades premium para grupos de consumo colaborativo e pequenas empresas.

Está também previsto o desenvolvimento de uma versão para iOS, com publicação na App Store, de forma a alargar o alcance da solução a um maior número de utilizadores.

9 Referências bibliográficas

- [AnDe] Android Developers, Android Developer Documentation,
<https://developer.android.com> , (website)
- [AnSt] Android Developers, Android Studio Documentation,
<https://developer.android.com/studio> , (website)
- [FiAu26] Firebase, Firebase Authentication Documentation,
<https://firebase.google.com/docs/auth> , (website)
- [Fire] Firebase, Firebase Documentation,
<https://firebase.google.com/docs> , (website)
- [Fist26] Firebase, Cloud Firestore Documentation,
<https://firebase.google.com/docs/firestore> , (website)
- [GoML] Google, ML Kit Documentation,
<https://developers.google.com/ml-kit> , (website)
- [GoMO25] Google, ML Kit Text Recognition Documentation,
<https://developers.google.com/ml-kit/vision/text-recognition> , (website)
- [Kotl21] JetBrains, Kotlin Documentation,
<https://kotlinlang.org/docs/home.html> , (website)
- [MaDe] Google, Material Design 3,
<https://m3.material.io> , (website)
- [NoWa25] NoWaste, NoWaste - Food Inventory App,
<https://www.nowasteapp.com> , (website)
- [OpFF26] Open Food Facts, Open Food Facts API Documentation,
<https://pt.openfoodfacts.org/data> , (website)
- [OpPF26] Open Products Facts, Open Products Facts API Documentation,
<https://world.openproductsfacts.org/data> , (website)
- [OuMi26] Out of Milk, Out of Milk App,
<https://www.outofmilk.com> , (website)
- [PaCh20] Pantry Check, Pantry Check App,
<https://pantrycheck.com> , (website)
- [UNat25] United Nations, Sustainable Development Goal 12: Responsible Consumption and Production,
<https://sdgs.un.org/goals/goal12> , (website)

Nota metodológica

Durante o desenvolvimento do presente projeto foram utilizadas ferramentas de inteligência artificial como apoio complementar à pesquisa de informação, clarificação de conceitos técnicos, organização de conteúdos e revisão textual. A sua utilização teve carácter auxiliar, não substituindo o trabalho de análise, modelação, implementação, validação e redação final, que foram integralmente realizados pelo autor.

Anexo 1 - Plano de Testes e Resultados

Este anexo apresenta o guião de testes realizado no âmbito do Projeto I, incluindo os cenários,

inputs, resultados esperados e resultados obtidos para os principais casos de uso da aplicação ReStock.

Tabela 12 - A1. Testes Unitários Automatizados

Ficheiro de Teste	Caso de Teste	Input	Resultado Esperado	Resultado Obtido
ModelIntegrityTest.kt	Validação de gamificação	User com 10 ações	Nível e pontos corretos	PASS
ModelIntegrityTest.kt	Consistência de entidades	User, Family, Product	Campos obrigatórios presentes	PASS
BudgetCalculationTest.kt	Total gasto no mês atual	Leite 1.5×2 + Pão 0.8×3	€5.40	PASS
BudgetCalculationTest.kt	Gastos por categoria	dairy + grocery products	dairy=5.0, grocery=2.4	PASS
BudgetCalculationTest.kt	Filtro de meses anteriores	Produto de mês-1	Excluído do cálculo	PASS
BudgetCalculationTest.kt	Formato da chave do snapshot	Data atual	YYYY-MM com zero à esquerda	PASS
CategoryUtilsTest.kt	Conversão PT → chave neutra	"Lacticínios"	"dairy"	PASS
CategoryUtilsTest.kt	Conversão EN → chave neutra	"Grocery"	"grocery"	PASS
CategoryUtilsTest.kt	Case insensitive	"DAIRY"	"dairy"	PASS
CategoryUtilsTest.kt	Categoria desconhecida	"desconhecida"	"desconhecida" (lowercase)	PASS
ProductModelTest.kt	Criação de produto com campos	id, nome, categoria, qtd	Campos corretamente atrib.	PASS
ProductModelTest.kt	Valores por omissão do produto	Product ()	nome="", qtd=0.0, unidade="un"	PASS

Tabela 13 - A2. Testes de Interface (Espresso)

Caso de Teste	Componente	Ação	Resultado Esperado	Resultado Obtido
Login com credenciais válidas	LoginActivity	Introduzir email + password válidos e clicar Login	Redirecionamento para HomeActivity	PASS
Login com credenciais inválidas	LoginActivity	Introduzir password errada	Mensagem de erro exibida	PASS
Renderização do ecrã de login	LoginActivity	Iniciar a Activity	Todos os campos visíveis	PASS
Google Sign-In disponível	LoginActivity	Verificar botão Google Sign-In	Botão presente e clicável	PASS

Tabela 14 - A3. Testes de Validação em Contexto Real

Cenário	Descrição	Resultado	Taxa de Sucesso
Sincronização familiar	Dois dispositivos com a mesma conta familiar atualizaram a lista de compras em simultâneo.	Atualizações refletidas em < 3 s	100%
Notificações de validade	Produto com validade há 2 dias gerou notificação push no dia configurado.	Notificação recebida corretamente	100%
Lista de compras partilhada	Quatro membros da família adicionaram/removeram itens durante uma semana sem conflitos.	Sem conflitos detetados	100%
Scanner de código de barras	Leitura de 20 produtos com código EAN-13 usando BarcodeScannerFragment + OpenFoodFacts.	19/20 produtos identificados	95%
OCR de fatura	Digitalização de fatura de supermercado com InvoiceScannerFragment + ML Kit Text Recognition.	Produtos principais detetados	90%
Robustez (ADB Monkey)	1 000 eventos aleatórios gerados pelo ADB Monkey sem crash da aplicação.	Sem crash registado	100%